
Python para Datos y tu Primer Data Warehouse Cloud

De Pandas a BigQuery: análisis profesional en la nube

Alex Goyzueta Delgado

alexgoyzueta2018@gmail.com

Python para Datos y tu Primer Data Warehouse Cloud

De Pandas a BigQuery: análisis profesional en la nube

© 2026 Alex Goyzueta Delgado

Todos los derechos reservados.

Primera edición: 2026

Colección **Data & Analytics Mastery** — Libro 2

Contacto: alexgoyzueta2018@gmail.com

Dedicatoria

A los que se atreven a migrar.

De la hoja de cálculo a la nube.

Del archivo local al data warehouse.

Del análisis estático al pipeline automatizado.

Prefacio

Colección Data & Analytics Mastery

Este es el **Libro 2** de la colección. En el Libro 1 aprendiste los fundamentos del analista de datos con SQL, Excel y Python. Ahora es el momento de dar el salto a la nube.

Qué aprenderás en este libro

1. **Python avanzado** — pandas, NumPy, visualización con Matplotlib y Seaborn
2. **BigQuery** — el data warehouse cloud de Google, desde cero hasta nivel profesional
3. **Looker Studio** — dashboards cloud conectados a datos en tiempo real
4. **Automatización** — pipelines serverless con Cloud Functions y Schedule Queries

Prerrequisitos

- Haber completado el Libro 1 o tener conocimientos equivalentes
- Conocer Python básico (variables, funciones, listas, diccionarios)
- Tener nociones de SQL (SELECT, JOIN, GROUP BY)
- Una cuenta gratuita de Google Cloud Platform (GCP)

Estructura del libro

4 proyectos que construyen sobre el anterior:

Proyecto 1: Python Avanzado	→ pandas, limpieza, visualización
Proyecto 2: BigQuery	→ cloud, data warehouse, SQL analítico
Proyecto 3: Looker Studio	→ dashboards cloud profesionales
Proyecto 4: Automatización	→ pipelines serverless

Material complementario

Todos los códigos y datasets están disponibles en la carpeta ``codigos/``.

Empieza instalando pandas y configurando tu cuenta de Google Cloud. En el Apéndice C encontrarás una guía paso a paso.

Introducción

De local a cloud: el salto del analista

En el Libro 1 trabajaste con datos locales: SQLite, Excel y Python en tu ordenador. En el mundo real, los datos empresariales no están en SQLite. Están en data warehouses cloud como BigQuery, Snowflake o Redshift.

Este libro te lleva de **analista local** a **analista cloud**.

Tecnologías que usarás

Tecnología	Propósito
Python + pandas	Análisis y transformación de datos
Matplotlib / Seaborn	Visualización de datos
Google Cloud Platform	Infraestructura cloud
BigQuery	Data warehouse serverless
Looker Studio	Dashboards cloud
Cloud Functions	Automatización serverless

El proyecto central: TechStore Cloud

Seguimos con TechStore, pero ahora migrado a la nube. Trabajarás con datasets más grandes, consultas más rápidas y dashboards compartidos.

Instalación inicial

```
pip install pandas numpy matplotlib seaborn pandas-gbq
```

Para BigQuery necesitarás:

1. Una cuenta de Google (gmail)
 2. Un proyecto en Google Cloud Console
 3. Facturación habilitada (usarás el tier gratuito)
- El Apéndice C cubre la configuración paso a paso.

Sobre el tier gratuito de GCP

Google Cloud ofrece:

- **\$300 USD** en créditos gratuitos por 90 días
- **BigQuery**: 10 GB de almacenamiento y 1 TB de consultas por mes gratis

- **Cloud Functions:** 2 millones de invocaciones gratis

Puedes completar todo el libro sin gastar dinero.

Capítulo 1: Repaso Rápido — Python y pandas para Análisis

De Python básico a pandas

En el Libro 1 aprendiste Python básico: variables, tipos, listas, diccionarios, funciones y bucles. Ahora vamos a dar el salto a **pandas**, la biblioteca más importante para análisis de datos en Python.

Si necesitas repasar Python básico, revisa el Apéndice B al final de este libro.

¿Qué es pandas?

pandas es una biblioteca que proporciona estructuras de datos rápidas y flexibles para trabajar con datos "relacionales" o "etiquetados". Sus dos estructuras principales son:

- **Series:** una columna de datos (como una lista con índice)
- **DataFrame:** una tabla completa (como una hoja de Excel o una tabla SQL)

```
import pandas as pd
import numpy as np

print(f"pandas versión: {pd.__version__}")
print(f"numpy versión: {np.__version__}")
```

Tu primer DataFrame

```
import pandas as pd

# Desde un diccionario
data = {
    "producto": ["Portátil Pro", "Smartphone Air", "Tablet Max"],
    "precio": [1299.99, 899.99, 499.99],
    "stock": [50, 120, 80],
    "categoria": ["Portátiles", "Smartphones", "Tablets"],
}
df = pd.DataFrame(data)
print(df)
```

producto	precio	stock	categoria
----------	--------	-------	-----------

0	Portátil Pro	1299.99	50	Portátiles
1	Smartphone Air	899.99	120	Smartphones
2	Tablet Max	499.99	80	Tablets

Desde CSV

```
df = pd.read_csv("codigos/datos_ventas.csv")
print(df.head()) # Primeras 5 filas
```

Exploración rápida de un DataFrame

```
# Dimensiones
print(f"Filas: {df.shape[0]}, Columnas: {df.shape[1]}")

# Información de columnas y tipos
print(df.info())

# Estadísticas descriptivas
print(df.describe())

# Nombres de columnas
print(df.columns.tolist())

# Valores únicos en una columna
print(df["region"].unique())
print(df["region"].value_counts())
```

Selección de datos

```
# Seleccionar una columna (devuelve una Serie)
totales = df["total"]

# Seleccionar múltiples columnas
df[["order_id", "total", "region"]]

# Filtrar filas
df[df["total"] > 1000]
df[df["region"] == "Madrid"]
df[(df["total"] > 500) & (df["region"] == "Cataluña")]

# Seleccionar por posición: .iloc
df.iloc[0] # Primera fila
df.iloc[1:5] # Filas 1 a 4
df.iloc[:, 0:3] # Columnas 0 a 2

# Seleccionar por etiqueta: .loc
df.loc[df["region"] == "Madrid", ["order_id", "total"]]
```

Trabajar con columnas

```
# Crear nueva columna
df["año"] = pd.to_datetime(df["order_date"]).dt.year
df["mes"] = pd.to_datetime(df["order_date"]).dt.month
df["ingreso_con_iva"] = df["total"] * 1.21

# Renombrar columnas
df = df.rename(columns={"total": "importe_total"})

# Eliminar columnas
df.drop(columns=["customer_name"], inplace=True)

# Reordenar columnas
df = df[["order_id", "order_date", "region", "total"]]
```

Valores nulos

```
# Detectar nulos
print(df.isnull().sum())

# Eliminar filas con nulos
df_sin_nulos = df.dropna()

# Rellenar nulos
df["employee_name"] = df["employee_name"].fillna("Sin asignar")
df["total"] = df["total"].fillna(0)
```

Guardar datos

```
# A CSV
df.to_csv("datos_limpios.csv", index=False)

# A Excel
df.to_excel("datos_limpios.xlsx", index=False, sheet_name="Ventas")

# A JSON
df.to_json("datos_limpios.json", orient="records")
```

Ejercicios del Capítulo 1

1. Importa pandas y numpy. Lee `datos\_ventas.csv` en un DataFrame.
2. Muestra las primeras 10 filas, la información del DataFrame y las estadísticas descriptivas.
3. ¿Cuántas filas y columnas tiene el DataFrame?
4. Filtra solo los pedidos de la región "Madrid" con total > 500.
5. Crea una columna "año" a partir de "order_date".
6. ¿Cuántos valores nulos hay en cada columna?
7. Agrupa por región y calcula el total de ventas (usa `groupby`).
8. Ordena el DataFrame por total descendente.
9. Guarda los pedidos completados en un archivo CSV separado.
10. Calcula el precio promedio por región usando `groupby`.

Checklist de autoevaluación

- ☐ Sé crear un DataFrame desde un diccionario y desde CSV
- ☐ Sé explorar un DataFrame con head, info, describe
- ☐ Sé seleccionar columnas y filtrar filas
- ☐ Sé crear y eliminar columnas
- ☐ Sé manejar valores nulos
- ☐ Sé guardar datos a CSV, Excel y JSON

Capítulo 2: pandas Avanzado — Merge, GroupBy, Pivot y Apply

Merge: el equivalente a JOINS de SQL

Igual que en SQL combinas tablas con JOINS, en pandas usas `merge()`.

```
import pandas as pd

ventas = pd.read_csv("codigos/datos_ventas.csv")
productos = pd.read_csv("codigos/datos_productos.csv")
clientes = pd.read_csv("codigos/datos_clientes.csv")

# INNER JOIN (solo registros que coinciden)
df = pd.merge(ventas, productos, left_on="customer_id", right_on="id")
```

Espera, eso no tiene sentido. Corrigiendo:

```
# INNER JOIN: ventas + productos por product_id
# Primero necesitamos detalles de ventas
detalles = pd.read_csv("codigos/datos_detalle_ventas.csv")
productos = pd.read_csv("codigos/datos_productos.csv")

df = pd.merge(detalles, productos, left_on="product_id", right_on="id", how="inner")
print(df.head())
print(f"Filas: {len(df)}")
```

Tipos de merge

```
# INNER JOIN (por defecto)
df = pd.merge(tabla1, tabla2, on="columna_comun")

# LEFT JOIN
df = pd.merge(tabla1, tabla2, on="columna_comun", how="left")

# RIGHT JOIN
df = pd.merge(tabla1, tabla2, on="columna_comun", how="right")

# FULL OUTER JOIN
df = pd.merge(tabla1, tabla2, on="columna_comun", how="outer")
```

Merge con múltiples columnas

```
# Merge con nombres de columna diferentes
ventas_con_cliente = pd.merge(
    ventas,
    clientes,
    left_on="customer_id",
    right_on="id",
    how="left"
)
```

GroupBy: agrupar y agregar

Equivalente a GROUP BY de SQL:

```
# Agrupar por región y calcular métricas
resumen = ventas.groupby("region").agg(
    pedidos=("order_id", "count"),
    ingresos=("total", "sum"),
    ticket_promedio=("total", "mean"),
    pedido_maximo=("total", "max"),
    pedido_minimo=("total", "min")
).reset_index()

print(resumen)
```

Múltiples niveles de agrupación

```
# Por región y estado
resumen = ventas.groupby(["region", "status"]).agg(
    pedidos=("order_id", "count"),
    ingresos=("total", "sum")
).reset_index()
```

Transformaciones con groupby

```
# Añadir columna con el promedio de la región
ventas["promedio_region"] = ventas.groupby("region")["total"].transform("mean")

# Añadir columna con el ranking dentro de la región
ventas["ranking_region"] = ventas.groupby("region")["total"].rank(ascending=False)
```

Pivot Table: tablas dinámicas

Equivalente a las tablas dinámicas de Excel:

```
# Tabla dinámica básica
pivot = pd.pivot_table(
    ventas,
    values="total",
    index="region",
    columns="status",
    aggfunc="sum",
    fill_value=0
)
```

```
)  
print(pivot)
```

Pivot table avanzada

```
pivot = pd.pivot_table(  
    ventas,  
    values="total",  
    index=["region", "customer_name"],  
    columns="status",  
    aggfunc={"total": "sum"},  
    fill_value=0,  
    margins=True,          # Añadir totales  
    margins_name="Total"  
)
```

Apply: aplicar funciones

`apply()` aplica una función a cada fila o columna:

```
# Función para clasificar pedidos  
def clasificar_pedido(total):  
    if total > 1000:  
        return "Alto valor"  
    elif total > 200:  
        return "Medio"  
    else:  
        return "Bajo"  
  
ventas["clasificacion"] = ventas["total"].apply(clasificar_pedido)  
print(ventas["clasificacion"].value_counts())
```

Apply con lambda

```
ventas["descuento_sugerido"] = ventas["total"].apply(  
    lambda x: x * 0.1 if x > 1000 else x * 0.05  
)
```

Apply en filas completas

```
def calcular_margen(fila):  
    return fila["total"] - fila["total"] * 0.6 # Margen estimado 40%  
  
ventas["margen_estimado"] = ventas.apply(calcular_margen, axis=1)
```

Operaciones con fechas

```
# Convertir a datetime  
ventas["order_date"] = pd.to_datetime(ventas["order_date"])  
  
# Extraer componentes
```

```

ventas["año"] = ventas["order_date"].dt.year
ventas["mes"] = ventas["order_date"].dt.month
ventas["día_semana"] = ventas["order_date"].dt.day_name()
ventas["semana"] = ventas["order_date"].dt.isocalendar().week

# Filtrar por fecha
ventas_2025 = ventas[ventas["order_date"].dt.year == 2025]

# Agrupar por mes
ventas["año_mes"] = ventas["order_date"].dt.to_period("M")
mensual = ventas.groupby("año_mes").agg(
    pedidos=("order_id", "count"),
    ingresos=("total", "sum")
)

```

Ejercicios del Capítulo 2

1. Combina `datos\_ventas.csv` con `datos\_clientes.csv` usando merge para añadir región y ciudad a cada pedido.
2. Agrupa por región y calcula: total de pedidos, ingresos totales, ticket promedio.
3. Crea una tabla dinámica con región en filas, status en columnas y suma de total como valores.
4. Usa apply para crear una columna "tamaño_pedido": Pequeño (<100), Medio (100-500), Grande (>500).
5. Añade una columna con el ranking de cada pedido dentro de su región por total descendente.
6. Calcula las ventas totales por año-mes usando groupby y .dt.to_period.
7. Filtra los pedidos del año 2025 y calcula el top 5 de clientes por gasto.
8. Usa transform para añadir a cada fila el promedio de su región.
9. Crea una tabla dinámica que muestre el promedio de ventas por región y día de la semana.
10. Usa merge para combinar detalles_ventas con productos y calcula el ingreso total por categoría.

Checklist de autoevaluación

- [] Sé combinar DataFrames con merge (INNER, LEFT, FULL)
- [] Sé agrupar datos con groupby y múltiples agregaciones
- [] Sé crear tablas dinámicas con pivot_table
- [] Sé usar apply y lambda para transformaciones
- [] Sé trabajar con fechas en pandas
- [] Sé usar transform para broadcasting de agregaciones

Capítulo 3: Limpieza de Datos Reales

El 80% del trabajo del analista

En el mundo real, los datos nunca llegan limpios. Vienen con:

- Valores nulos donde no debería haberlos
- Formatos inconsistentes (fechas, monedas, texto)
- Duplicados
- Valores atípicos (outliers)
- Errores de tipeo o carga

Este capítulo te enseña a detectar y solucionar cada problema.

Cargar datos sucios

Vamos a crear un dataset con problemas reales para practicar:

```
import pandas as pd
import numpy as np

# Dataset con problemas
data = {
    "cliente_id": [1, 2, 3, 4, 5, 5, 6, 7, 8, 9],
    "nombre": ["Ana García", "Luis Pérez", "María López", "Juan Martínez",
               None, "Carlos Ruiz", "Elena Sánchez", "Pedro Gómez", None, "Sofía Díaz"],
    "email": ["ana@email.com", "luis@email", "maria@email.com", None,
              "carlos@email.com", "carlos@email.com", "elena@email.com",
              "pedro@email.com", None, "sofia@email.com"],
    "fecha_registro": ["2024-01-15", "2024/03/20", "15-01-2024", "2024-06-01",
                       "2024-07-01", "2024-07-01", "2024-08-15", "2024-09-01",
                       "enero 2025", "2024-11-15"],
    "gasto_total": [1250.50, 899.99, None, 450.00, 2300.00, 2300.00, -50.00, 999999, 350.00, 780.00],
    "edad": [32, 28, None, 45, 38, 38, 200, 29, 42, 31],
}

df = pd.DataFrame(data)
print("DATOS CRUDOS:")
print(df)
print(f"\nDimensiones: {df.shape}")
```

1. Valores nulos (missing data)

```
# Detectar nulos
print("Nulos por columna:")
print(df.isnull().sum())
print(f"\nTotal nulos: {df.isnull().sum().sum()}")
print(f"% nulos por columna:\n{df.isnull().mean() * 100}")
```

Estrategias para manejar nulos

```
# 1. Eliminar filas con nulos (si son pocos)
df_sin_nulos = df.dropna()

# 2. Eliminar columnas con demasiados nulos
df = df.drop(columns=["nombre"]) # Si la columna no es crítica

# 3. Rellenar con un valor constante
df["nombre"] = df["nombre"].fillna("Cliente desconocido")
df["email"] = df["email"].fillna("sin-email@pendiente.com")

# 4. Rellenar con la media o mediana
df["gasto_total"] = df["gasto_total"].fillna(df["gasto_total"].median())

# 5. Rellenar con el valor anterior (forward fill)
df["edad"] = df["edad"].fillna(method="ffill")
```

2. Duplicados

```
# Detectar duplicados
print(f"Filas duplicadas: {df.duplicated().sum()}")
print(f"Filas duplicadas (por cliente_id): {df.duplicated(subset=['cliente_id']).sum()}")

# Ver duplicados
print(df[df.duplicated(subset="cliente_id", keep=False)])

# Eliminar duplicados
df = df.drop_duplicates(subset=["cliente_id", keep="first"])
print(f"Después de limpiar duplicados: {len(df)} filas")
```

3. Formatos inconsistentes

Fechas

```
# Intentar convertir a datetime (detecta errores)
df["fecha_registro"] = pd.to_datetime(df["fecha_registro"], errors="coerce")
print("Fechas después de conversión:")
print(df["fecha_registro"].head(10))

# Las fechas que no pudo convertir se vuelven NaT (Not a Time)
print(f"Fechas inválidas: {df['fecha_registro'].isna().sum()}")
```

Texto

```
# Limpiar texto
df["nombre"] = df["nombre"].str.strip() # Quitar espacios
df["email"] = df["email"].str.strip().str.lower()

# Validar emails (que contengan @ y .)
emails_validos = df["email"].str.contains(r"^[^@]+\.[^@]+$", na=False)
print(f"Emails inválidos: {(~emails_validos).sum()}")
df["email"] = df["email"].where(emails_validos, "email-invalido@corregir.com")
```

4. Valores atípicos (outliers)

```
# Detectar outliers con el rango intercuartílico (IQR)
Q1 = df["gasto_total"].quantile(0.25)
Q3 = df["gasto_total"].quantile(0.75)
IQR = Q3 - Q1

limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

print(f"Límite inferior: {limite_inferior:.2f}")
print(f"Límite superior: {limite_superior:.2f}")

outliers = df[(df["gasto_total"] < limite_inferior) | (df["gasto_total"] > limite_superior)]
print(f"Outliers detectados: {len(outliers)}")
print(outliers[["cliente_id", "gasto_total"]])

# Manejo de outliers:
# Opción 1: Eliminar
df = df[(df["gasto_total"] >= limite_inferior) & (df["gasto_total"] <= limite_superior)]

# Opción 2: Limitar (capping)
df["gasto_total"] = df["gasto_total"].clip(lower=0, upper=df["gasto_total"].quantile(0.95))

# Opción 3: Reemplazar con la mediana
mediana = df["gasto_total"].median()
df.loc[df["gasto_total"] > limite_superior, "gasto_total"] = mediana
```

Pipeline de limpieza completo

```
def limpiar_ventas(df):
    """Pipeline completo de limpieza"""
    df = df.copy()

    # 1. Eliminar duplicados
    df = df.drop_duplicates(subset=["order_id"])

    # 2. Convertir fechas
    df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce")

    # 3. Eliminar filas con fecha inválida
```



```

df = df.dropna(subset=["order_date"])

# 4. Rellenar nulos en columnas de texto
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].fillna("Sin dato")

# 5. Rellenar nulos en columnas numéricas con la mediana
for col in df.select_dtypes(include="number").columns:
    df[col] = df[col].fillna(df[col].median())

# 6. Eliminar outliers en total
Q1 = df["total"].quantile(0.25)
Q3 = df["total"].quantile(0.75)
IQR = Q3 - Q1
df = df[(df["total"] >= Q1 - 1.5 * IQR) & (df["total"] <= Q3 + 1.5 * IQR)]

# 7. Estandarizar texto
df["region"] = df["region"].str.strip().str.title()

return df

# Aplicar pipeline
ventas_limpias = limpiar_ventas(pd.read_csv("codigos/datos_ventas.csv"))
print(f"Antes: 6000 filas | Después: {len(ventas_limpias)} filas")

```

Ejercicios del Capítulo 3

1. Carga el dataset de ejemplo con problemas (data del inicio del capítulo).
2. Identifica cuántos valores nulos hay en cada columna.
3. Decide qué estrategia usarías para cada columna con nulos y aplícala.
4. Encuentra y elimina filas duplicadas por cliente_id.
5. Corrige las fechas con formato inconsistente.
6. Detecta outliers en gasto_total usando IQR y explícalos.
7. ¿Por qué una edad de 200 años es un outlier? ¿Qué harías con ella?
8. Crea una función `limpiar\_clientes()` que limpie el dataset de clientes.
9. Aplica el pipeline de limpieza al archivo `datos\_ventas.csv` real.
10. Guarda el resultado limpio en un nuevo archivo CSV.

Checklist de autoevaluación

- ☐ Sé detectar y manejar valores nulos
- ☐ Sé identificar y eliminar duplicados
- ☐ Sé limpiar formatos inconsistentes (fechas, texto)
- ☐ Sé detectar outliers con IQR
- ☐ Sé crear un pipeline de limpieza reutilizable
- ☐ Entiendo que la limpieza es específica para cada dataset

Capítulo 4: Visualización con Matplotlib y Seaborn

Por qué visualizar

Un gráfico bien diseñado comunica en segundos lo que una tabla tarda minutos. En análisis de datos, la visualización es tu herramienta más poderosa para:

1. **Explorar** datos (EDA)
2. **Comunicar** hallazgos
3. **Detectar** patrones y anomalías

Matplotlib: el clásico

Matplotlib es la biblioteca fundacional de visualización en Python. Es flexible y potente, pero requiere más código.

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

ventas = pd.read_csv("codigos/datos_ventas.csv")
```

Gráfico de líneas

```
# Ventas mensuales
ventas["order_date"] = pd.to_datetime(ventas["order_date"])
ventas["mes"] = ventas["order_date"].dt.to_period("M")
mensual = ventas.groupby("mes")["total"].sum().reset_index()
mensual["mes"] = mensual["mes"].astype(str)

plt.figure(figsize=(12, 6))
plt.plot(mensual["mes"], mensual["total"], marker="o", linewidth=2, color="#2E75B6")
plt.title("Evolución Mensual de Ventas", fontsize=16, pad=20)
plt.xlabel("Mes", fontsize=12)
plt.ylabel("Ingresos (€)", fontsize=12)
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Gráfico de barras

```
# Ventas por región
region = ventas.groupby("region")["total"].sum().sort_values(ascending=False)

plt.figure(figsize=(10, 6))
colors = ["#2E75B6", "#4BACC6", "#F4B183", "#A9D18E", "#D6A3E4"]
bars = plt.bar(region.index, region.values, color=colors[:len(region)], edgecolor="white")

# Añadir etiquetas
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height,
             f"{height:,.0f}€", ha="center", va="bottom", fontsize=10)

plt.title("Ventas por Región", fontsize=16, pad=20)
plt.xlabel("Región", fontsize=12)
plt.ylabel("Ingresos (€)", fontsize=12)
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

Histograma

```
# Distribución del ticket
plt.figure(figsize=(10, 6))
plt.hist(ventas["total"], bins=50, color="#2E75B6", edgecolor="white", alpha=0.8)
plt.title("Distribución del Ticket Promedio", fontsize=16, pad=20)
plt.xlabel("Total del pedido (€)", fontsize=12)
plt.ylabel("Frecuencia", fontsize=12)
plt.axvline(ventas["total"].mean(), color="red", linestyle="--", label=f"Media: {ventas['total'].mean():.2f}")
plt.axvline(ventas["total"].median(), color="green", linestyle="--", label=f"Mediana: {ventas['total'].median():.2f}")
plt.legend()
plt.tight_layout()
plt.show()
```

Boxplot

```
# Distribución por región
plt.figure(figsize=(12, 6))
regiones = [ventas[ventas["region"] == r]["total"] for r in ventas["region"].unique()]
plt.boxplot(regiones, labels=ventas["region"].unique())
plt.title("Distribución de Ventas por Región", fontsize=16, pad=20)
plt.ylabel("Total del pedido (€)", fontsize=12)
plt.xticks(rotation=30)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Seaborn: visualización estadística

Seaborn está construido sobre matplotlib y ofrece gráficos estadísticos con menos código.

```
import seaborn as sns
sns.set_theme(style="whitegrid")
```

Barplot con Seaborn

```
plt.figure(figsize=(10, 6))
sns.barplot(data=ventas, x="region", y="total", estimator=sum, ci=None,
            palette="Blues_d", order=ventas.groupby("region")["total"].sum().sort_values(ascending=False).
plt.title("Ventas Totales por Región", fontsize=16)
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

Boxplot con Seaborn

```
plt.figure(figsize=(12, 6))
sns.boxplot(data=ventas, x="region", y="total", palette="Set2")
plt.title("Distribución de Ventas por Región", fontsize=16)
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

Distribuciones

```
# Histograma + KDE
plt.figure(figsize=(10, 6))
sns.histplot(ventas["total"], bins=50, kde=True, color="#2E75B6")
plt.title("Distribución del Valor de Pedidos", fontsize=16)
plt.xlabel("Total (€)")
plt.tight_layout()
plt.show()
```

Pairplot (matriz de correlaciones)

```
# Solo para columnas numéricas
num_cols = ventas.select_dtypes(include=np.number).columns[:4]
sns.pairplot(ventas[num_cols], diag_kind="kde")
plt.show()
```

Heatmap de correlaciones

```
# Matriz de correlación
corr = ventas.select_dtypes(include=np.number).corr()

plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, cmap="coolwarm", center=0, fmt=".2f",
            square=True, linewidths=1, cbar_kws={"shrink": 0.8})
plt.title("Matriz de Correlaciones", fontsize=16, pad=20)
plt.tight_layout()
plt.show()
```

Countplot

```
plt.figure(figsize=(8, 5))
sns.countplot(data=ventas, x="status", palette="Set2",
              order=ventas["status"].value_counts().index)
plt.title("Distribución de Estados de Pedido", fontsize=16)
plt.tight_layout()
plt.show()
```

Personalización avanzada

```
# Estilo profesional global
plt.style.use("seaborn-v0_8-whitegrid")
sns.set_palette("Blues_d")

# Configuración global
plt.rcParams.update({
    "figure.figsize": (12, 6),
    "font.size": 12,
    "axes.titlesize": 16,
    "axes.labelsize": 12,
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "figure.dpi": 150,
})

# Guardar gráficos
plt.savefig("grafico.png", dpi=300, bbox_inches="tight")
plt.savefig("grafico.pdf", bbox_inches="tight")
```

Dashboard de 4 gráficos

```
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# 1. Ventas por región
region = ventas.groupby("region")["total"].sum().sort_values()
axes[0, 0].barh(region.index, region.values, color="#2E75B6")
axes[0, 0].set_title("Ventas por Región")

# 2. Evolución mensual
mensual = ventas.groupby("mes")["total"].sum()
axes[0, 1].plot(mensual.index.astype(str), mensual.values, marker="o", color="#2E75B6")
axes[0, 1].set_title("Tendencia Mensual")
axes[0, 1].tick_params(axis="x", rotation=45)

# 3. Distribución de pedidos
axes[1, 0].hist(ventas["total"], bins=30, color="#4BACC6", edgecolor="white")
axes[1, 0].set_title("Distribución de Pedidos")

# 4. Estado de pedidos
status = ventas["status"].value_counts()
axes[1, 1].pie(status.values, labels=status.index, autopct="%1.1f%%",
```

```
colors=["#2E75B6", "#4BACC6", "#F4B183", "#A9D18E"]  
axes[1, 1].set_title("Estado de Pedidos")  
  
plt.tight_layout()  
plt.show()
```

Ejercicios del Capítulo 4

1. Crea un gráfico de barras con las ventas totales por región usando matplotlib.
2. Crea un histograma de la distribución del total de pedidos.
3. Usa seaborn para crear un boxplot de ventas por región.
4. Crea un heatmap de correlaciones entre columnas numéricas.
5. Genera un gráfico de líneas con la evolución mensual de ventas.
6. Crea un countplot de los estados de pedido.
7. Diseña un dashboard de 2x2 con 4 gráficos diferentes.
8. Guarda un gráfico en PNG con 300 DPI.
9. Personaliza los colores y el estilo del gráfico de barras.
10. Crea un pairplot con 4 variables numéricas y analiza las correlaciones visuales.

Checklist de autoevaluación

- ☐ Sé crear gráficos de líneas, barras y histogramas con matplotlib
- ☐ Sé crear boxplots, heatmaps y pairplots con seaborn
- ☐ Sé personalizar colores, etiquetas y estilos
- ☐ Sé crear dashboards de múltiples gráficos
- ☐ Sé guardar gráficos en alta resolución
- ☐ Entiendo cuándo usar cada tipo de gráfico

Capítulo 5: Análisis Exploratorio de Datos (EDA) Completo

¿Qué es el EDA?

El Análisis Exploratorio de Datos (EDA) es el proceso de investigar un dataset para descubrir patrones, anomalías, relaciones y tendencias. No es un paso único: es un ciclo iterativo de:

1. **Preguntar** — ¿Qué quiero saber?
2. **Visualizar** — ¿Qué muestran los datos?
3. **Transformar** — ¿Necesito limpiar o crear nuevas variables?
4. **Concluir** — ¿Qué aprendí?

EDA completo de TechStore

Vamos a realizar un EDA profesional sobre los datos de ventas de TechStore.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
plt.rcParams.update({"figure.dpi": 150})

# Cargar datos
ventas = pd.read_csv("codigos/datos_ventas.csv")
clientes = pd.read_csv("codigos/datos_clientes.csv")
detalles = pd.read_csv("codigos/datos_detalle_ventas.csv")
productos = pd.read_csv("codigos/datos_productos.csv")

print("=" * 60)
print("EDA COMPLETO - TECHSTORE")
print("=" * 60)
```

1. Visión general del dataset

```
def overview(df, name):
    print(f"\n--- {name} ---")
    print(f"Dimensiones: {df.shape}")
```

```

print(f"\nColumnas: {df.columns.tolist()}")
print(f"\nTipos:\n{df.dtypes}")
print(f"\nPrimeras filas:")
print(df.head(3))
print(f"\nNulos:\n{df.isnull().sum()}")
print(f"\nEstadísticas:\n{df.describe(include='all')}")

overview(ventas, "VENTAS")
overview(clientes, "CLIENTES")
overview(productos, "PRODUCTOS")

```

2. Análisis univariante

```

# Configurar dashboard de distribuciones
fig, axes = plt.subplots(2, 3, figsize=(16, 10))

# 1. Distribución del total de pedidos
axes[0, 0].hist(ventas["total"], bins=50, color="#2E75B6", edgecolor="white")
axes[0, 0].set_title("Distribución del Total de Pedidos")
axes[0, 0].set_xlabel("Total (€)")
axes[0, 0].axvline(ventas["total"].mean(), color="red", ls="--", label=f"Media: {ventas['total'].mean():.0f}")
axes[0, 0].legend()

# 2. Distribución de pedidos por estado
status_counts = ventas["status"].value_counts()
axes[0, 1].bar(status_counts.index, status_counts.values, color=["#2E75B6", "#4BACC6", "#F4B183", "#A9D18E"])
axes[0, 1].set_title("Pedidos por Estado")
axes[0, 1].tick_params(axis="x", rotation=30)

# 3. Top 10 regiones por pedidos
region_counts = ventas["region"].value_counts().head(10)
axes[0, 2].barh(region_counts.index[:10], region_counts.values[:10], color="#2E75B6")
axes[0, 2].set_title("Top 10 Regiones por Pedidos")

# 4. Distribución de ventas por año
ventas["order_date"] = pd.to_datetime(ventas["order_date"])
ventas["año"] = ventas["order_date"].dt.year
año_ventas = ventas.groupby("año")["total"].sum()
axes[1, 0].bar(año_ventas.index, año_ventas.values, color="#4BACC6")
axes[1, 0].set_title("Ventas por Año")
axes[1, 0].set_xlabel("Año")
axes[1, 0].set_ylabel("Ingresos (€)")

# 5. Top 10 clientes por gasto
top_clientes = ventas.groupby("customer_name")["total"].sum().sort_values(ascending=False).head(10)
axes[1, 1].barh(top_clientes.index[:10], top_clientes.values[:10], color="#F4B183")
axes[1, 1].set_title("Top 10 Clientes por Gasto")

# 6. Distribución de pedidos por mes
ventas["mes"] = ventas["order_date"].dt.month
mes_ventas = ventas.groupby("mes")["total"].sum()

```



```

axes[1, 2].plot(mes_ventas.index, mes_ventas.values, marker="o", color="#2E75B6", linewidth=2)
axes[1, 2].set_title("Ventas por Mes (estacionalidad)")
axes[1, 2].set_xlabel("Mes")
axes[1, 2].set_xticks(range(1, 13))

plt.tight_layout()
plt.show()

```

3. Análisis bivalente

```

# Relaciones entre variables

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Ventas por región y estado
pivot = pd.pivot_table(ventas, values="total", index="region", columns="status", aggfunc="sum", fill_value=0)
pivot.plot(kind="bar", ax=axes[0, 0])
axes[0, 0].set_title("Ventas por Región y Estado")
axes[0, 0].legend(loc="upper right")
axes[0, 0].tick_params(axis="x", rotation=45)

# 2. Ticket promedio por región
ticket_region = ventas.groupby("region")["total"].mean().sort_values()
axes[0, 1].barh(ticket_region.index, ticket_region.values, color="#4BACC6")
axes[0, 1].set_title("Ticket Promedio por Región")
axes[0, 1].set_xlabel("Ticket promedio (€)")

# 3. Boxplot: distribución por región
sns.boxplot(data=ventas, x="region", y="total", ax=axes[1, 0], palette="Set2")
axes[1, 0].set_title("Distribución de Ventas por Región")
axes[1, 0].tick_params(axis="x", rotation=45)

# 4. Ventas por día de la semana
ventas["día_semana"] = ventas["order_date"].dt.day_name()
orden_dias = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
dias_ventas = ventas.groupby("día_semana")["total"].sum().reindex(orden_dias)
axes[1, 1].plot(dias_ventas.index, dias_ventas.values, marker="s", color="#F4B183", linewidth=2)
axes[1, 1].set_title("Ventas por Día de la Semana")
axes[1, 1].tick_params(axis="x", rotation=30)

plt.tight_layout()
plt.show()

```

4. Análisis de correlaciones

```

# Para análisis de correlación, combinamos tablas
detalles_con_productos = pd.merge(detalles, productos, left_on="product_id", right_on="id")
detalles_con_productos["line_total"] = detalles_con_productos["quantity"] * detalles_con_productos["unit_price"]

# Seleccionar columnas numéricas

```

```

num_cols = detalles_con_productos.select_dtypes(include=np.number).columns

# Matriz de correlación
corr = detalles_con_productos[num_cols].corr()

plt.figure(figsize=(12, 10))
mask = np.triu(corr)
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0,
            square=True, mask=mask, linewidths=1, cbar_kws={"shrink": 0.8})
plt.title("Matriz de Correlaciones – Detalles de Ventas", fontsize=16, pad=20)
plt.tight_layout()
plt.show()

```

5. Segmentación de clientes (RFM básico)

```

# RFM: Recencia, Frecuencia, Valor Monetario
from datetime import datetime

fecha_referencia = datetime(2026, 7, 1)

rfm = ventas.groupby("customer_id").agg(
    recencia=("order_date", lambda x: (fecha_referencia - pd.to_datetime(x).max()).days),
    frecuencia=("order_id", "count"),
    monetario=("total", "sum")
).reset_index()

# Segmentar
rfm["segmento_recencia"] = pd.qcut(rfm["recencia"], q=4, labels=["Alta", "Media-Alta", "Media-Baja", "Baja"])
rfm["segmento_frecuencia"] = pd.qcut(rfm["frecuencia"], q=4, labels=["Baja", "Media-Baja", "Media-Alta", "Alta"])
rfm["segmento_valor"] = pd.qcut(rfm["monetario"], q=4, labels=["Bajo", "Medio-Bajo", "Medio-Alto", "Alto"])

# Puntaje combinado
rfm["puntaje_rfm"] = (
    rfm["segmento_recencia"].astype(str) + " | " +
    rfm["segmento_frecuencia"].astype(str) + " | " +
    rfm["segmento_valor"].astype(str)
)

print("Top 10 clientes por valor:")
print(rfm.sort_values("monetario", ascending=False).head(10))

```

6. Informe ejecutivo

```

print("\n" + "=" * 60)
print("INFORME EJECUTIVO – TECHSTORE")
print("=" * 60)

print(f"\n MÉTRICAS PRINCIPALES:")
print(f" • Ingresos totales: {ventas['total'].sum():.2f}€")
print(f" • Total pedidos: {len(ventas):,}")

```

```

print(f" • Clientes únicos: {ventas['customer_id'].nunique():,}")
print(f" • Ticket promedio: {ventas['total'].mean():,.2f}€")
print(f" • Pedido máximo: {ventas['total'].max():,.2f}€")
print(f" • Pedido mínimo: {ventas['total'].min():,.2f}€")

print(f"\n TENDENCIAS:")
print(f" • Región con más ventas: {ventas.groupby('region')['total'].sum().idxmax()}")
print(f" • Mes con más ventas: {ventas.groupby('mes')['total'].sum().idxmax()}")
print(f" • Día con más ventas: {ventas.groupby('día_semana')['total'].sum().idxmax()}")

print(f"\n ANOMALÍAS:")
print(f" • Pedidos cancelados: {len(ventas[ventas['status'] == 'Cancelado']):,}")
print(f" • % cancelación: {len(ventas[ventas['status'] == 'Cancelado'])/len(ventas)*100:.1f}%")

print(f"\n RECOMENDACIONES:")
print(f" • Enfocar marketing en regiones con bajo ticket promedio")
print(f" • Investigar causa de cancelaciones en meses altos")
print(f" • Programa de fidelización para clientes con alta frecuencia")

```

Ejercicios del Capítulo 5

1. Carga los datos de TechStore y realiza una visión general de cada tabla.
2. Crea un dashboard de 6 gráficos para el análisis univariante.
3. Analiza la relación entre región y ticket promedio.
4. Calcula la matriz de correlación entre precio, cantidad y total.
5. Realiza un análisis RFM básico de los clientes.
6. ¿Qué día de la semana tiene más ventas? ¿Y menos?
7. ¿Hay estacionalidad en las ventas? ¿Qué meses destacan?
8. ¿Qué región tiene la menor tasa de cancelación?
9. Genera un informe ejecutivo con las 5 métricas principales.
10. Escribe 3 recomendaciones basadas en tu EDA.

Checklist de autoevaluación

- ☐ Sé realizar un EDA completo: univariante, bivariate, correlaciones
- ☐ Sé crear dashboards de múltiples gráficos
- ☐ Sé hacer análisis RFM básico
- ☐ Sé generar un informe ejecutivo con hallazgos
- ☐ Sé formular recomendaciones basadas en datos
- ☐ Entiendo el flujo completo del EDA

Checkpoint 1: EDA Completo de TechStore

Objetivo

Has completado los cinco capítulos del Proyecto 1: Python avanzado, pandas, limpieza de datos, visualización y EDA. Ahora es momento de aplicar todo en un **análisis exploratorio completo de TechStore**.

Requisitos

- Tener `techstore.db` en la carpeta `codigos/`
- Tener instaladas las bibliotecas: pandas, numpy, matplotlib, seaborn, sqlite3
- Haber completado los capítulos 1 al 5

Paso 1: Carga y preparación de datos

Crea un script `codigos/eda\_completo\_techstore.py`:

```
import pandas as pd
import numpy as np
import sqlite3
import matplotlib.pyplot as plt
import seaborn as sns

conn = sqlite3.connect("../codigos/techstore.db")

# Cargar todas las tablas
orders = pd.read_sql_query("SELECT * FROM orders", conn)
customers = pd.read_sql_query("SELECT * FROM customers", conn)
products = pd.read_sql_query("SELECT * FROM products", conn)
order_items = pd.read_sql_query("SELECT * FROM order_items", conn)
employees = pd.read_sql_query("SELECT * FROM employees", conn)

print("=== VISTA GENERAL ===")
for name, df in [("orders", orders), ("customers", customers),
                 ("products", products), ("order_items", order_items),
                 ("employees", employees)]:
    print(f"{name}: {df.shape[0]} filas, {df.shape[1]} columnas")
```

```
conn.close()
```

Paso 2: Análisis univariante

```
# Ventas por mes
orders["order_date"] = pd.to_datetime(orders["order_date"])
orders["mes"] = orders["order_date"].dt.to_period("M")

fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# Histograma de totales
orders["total"].hist(ax=axes[0, 0], bins=30, color="steelblue", edgecolor="white")
axes[0, 0].set_title("Distribución de Totales")

# Boxplot de totales
orders.boxplot(column="total", ax=axes[0, 1])
axes[0, 1].set_title("Boxplot de Totales")

# Pedidos por estado
orders["status"].value_counts().plot(ax=axes[0, 2], kind="bar", color="coral")
axes[0, 2].set_title("Pedidos por Estado")

# Clientes por ciudad
customers["city"].value_counts().head(10).plot(ax=axes[1, 0], kind="barh", color="teal")
axes[1, 0].set_title("Top 10 Ciudades")

# Productos por categoría
products["category"].value_counts().plot(ax=axes[1, 1], kind="pie", autopct="%1.1f%%")
axes[1, 1].set_title("Productos por Categoría")

# Empleados por cargo
employees["position"].value_counts().plot(ax=axes[1, 2], kind="bar", color="purple")
axes[1, 2].set_title("Empleados por Cargo")

plt.tight_layout()
plt.savefig("eda_univariante.png", dpi=150)
plt.show()
```

Paso 3: Análisis bivalente

```
# Merge orders + customers
df = orders.merge(customers, on="customer_id")

# Ticket promedio por ciudad
top_ciudades = df.groupby("city").agg(
    pedidos=("order_id", "count"),
    ingresos=("total", "sum"),
    ticket_promedio=("total", "mean")
).sort_values("ingresos", ascending=False).head(10)
print("\nTop 10 ciudades por ingresos:")
```

```

print(top_ciudades)

# Merge order_items + products
df_items = order_items.merge(products, on="product_id")

# Ingresos por categoría
ingresos_cat = df_items.groupby("category").apply(
    lambda x: (x["quantity"] * x["unit_price"]).sum()
).sort_values(ascending=False)
print("\nIngresos por categoría:")
print(ingresos_cat)

# Correlación entre precio unitario y cantidad
print(f"\nCorrelación precio-cantidad: {df_items['unit_price'].corr(df_items['quantity']):.3f}")

```

Paso 4: Dashboard ejecutivo

```

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Evolución de ingresos mensuales
orders["order_date"] = pd.to_datetime(orders["order_date"])
orders.set_index("order_date").resample("ME")["total"].sum().plot(
    ax=axes[0, 0], marker="o", title="Ingresos Mensuales", color="green")

# 2. Top 10 productos
top_productos = df_items.groupby("name").apply(
    lambda x: (x["quantity"] * x["unit_price"]).sum()
).sort_values(ascending=False).head(10)
top_productos.plot(ax=axes[0, 1], kind="barh", title="Top 10 Productos", color="navy")

# 3. Mapa de calor: pedidos por día y mes
orders["dia_semana"] = orders["order_date"].dt.dayofweek
orders["mes"] = orders["order_date"].dt.month
heatmap_data = orders.pivot_table(index="dia_semana", columns="mes",
                                   values="order_id", aggfunc="count")
sns.heatmap(heatmap_data, ax=axes[1, 0], cmap="YlOrRd", annot=True, fmt=".0f")
axes[1, 0].set_title("Pedidos por Día y Mes")

# 4. Clientes por cantidad de pedidos
frecuencia_cliente = orders.groupby("customer_id").size()
frecuencia_cliente.value_counts().sort_index().plot(
    ax=axes[1, 1], kind="bar", title="Frecuencia de Pedidos por Cliente",
    color="orange", width=0.8)
axes[1, 1].set_xlabel("Número de pedidos")
axes[1, 1].set_ylabel("Clientes")

plt.tight_layout()
plt.savefig("eda_dashboard.png", dpi=150)
plt.show()

```

Paso 5: Conclusiones y recomendaciones

Basado en tu EDA, documenta:

1. **Métricas clave:** ingresos totales, ticket promedio, clientes únicos, pedidos totales
2. **Patrones:** estacionalidad, días pico, categorías estrella
3. **Problemas:** datos faltantes, outliers, anomalías
4. **Recomendaciones:** 3 acciones concretas basadas en datos
5. **Próximos pasos:** qué más te gustaría analizar

Entregables del Checkpoint 1

- [] Script ``eda_completo_techstore.py`` funcionando
- [] Gráfico de análisis univariante (6 paneles)
- [] Análisis bivariante (top ciudades, categorías, correlaciones)
- [] Dashboard ejecutivo (4 gráficos)
- [] Documento con conclusiones y recomendaciones

¡Felicidades! Has completado el análisis exploratorio de TechStore. En el Proyecto 2 migrarás estos datos a la nube con BigQuery.

Capítulo 6: Cloud Computing — Conceptos Fundamentales

Del escritorio a la nube

Hasta ahora hemos trabajado con Python, SQL y pandas en tu máquina local. Todo corre en tu CPU, usa tu RAM y guarda archivos en tu disco duro. Esto funciona bien con datasets pequeños (TechStore: ~6000 pedidos, ~500 productos), pero ¿qué pasa cuando tienes millones de filas, terabytes de datos, o necesitas compartir resultados con tu equipo en tiempo real?

Ahí entra la **nube** (cloud computing).

En este proyecto vas a migrar TechStore desde tu SQLite local a **BigQuery**, el data warehouse en la nube de Google Cloud Platform. Pero antes de escribir consultas, necesitas entender cómo piensa la nube.

¿Qué es cloud computing?

Cloud computing es la entrega de recursos informáticos (servidores, almacenamiento, bases de datos, redes, software) a través de internet, bajo demanda y con pago por uso.

En lugar de comprar y mantener tus propios servidores, alquilas lo que necesitas de un proveedor como Google Cloud, AWS o Azure.

Modelo de responsabilidad compartida

Tú eres responsable de:	El proveedor es responsable de:
Tus datos	Hardware físico
Tu código	Redes
Configuraciones de seguridad	Virtualización
IAM (accesos)	Electricidad y refrigeración
	Seguridad física del datacenter

Tipos de servicio en la nube

Modelo	Siglas	Ejemplo	Tú gestionas
Infrastructure as a Service	IaaS	Compute Engine (VM)	SO, runtime, datos
Platform as a Service	PaaS	BigQuery, App Engine	Solo datos y código
Software as a Service	SaaS	Gmail, Looker Studio	Solo usar la app

Function as a Service	FaaS	Cloud Functions	Solo una función
-----------------------	------	-----------------	------------------

BigQuery es **PaaS**: no gestionas servidores ni discos. Solo subes datos y escribes SQL. Google se encarga del escalado y mantenimiento.

Ventajas clave de la nube

1. **Escalado elástico**: de 0 a miles de servidores en segundos
2. **Pago por uso**: solo pagas lo que usas (storage + compute separados)
3. **Disponibilidad global**: datacenters en todo el mundo
4. **Mantenimiento cero**: el proveedor actualiza parches y hardware
5. **Seguridad**: inversión masiva en certificaciones y protecciones

Desventajas y consideraciones

- **Costes impredecibles** si no monitorizas
- **Dependencia de internet**: sin conexión, sin acceso
- **Vendor lock-in**: migrar entre proveedores es complejo
- **Privacidad de datos**: según la legislación del país del datacenter

> **Regla práctica**: usa la nube cuando necesites escalar, compartir, o procesar datos que no caben en tu máquina. Para proyectos personales pequeños, local sigue siendo una opción válida.

Almacenamiento en la nube: Object Storage vs Data Warehouse vs Data Lake

Tipo	Uso	Ejemplo GCP
Object Storage	Archivos sin procesar (CSVs, JSONs, imágenes)	Cloud Storage (GCS)
Data Warehouse	Datos estructurados para análisis y reportes	BigQuery
Data Lake	Datos brutos en formato nativo (estructurados y no)	Dataproc / GCS
Database	Transacciones OLTP (insert/update/delete rápidos)	Cloud SQL, Firestore

TechStore empezó como SQLite (database local). Ahora lo moverás a BigQuery (data warehouse cloud) para análisis escalable.

Cloud Storage (GCS) — El sistema de archivos de la nube

Antes de BigQuery, necesitas un lugar para subir tus archivos. Cloud Storage organiza los datos en **buckets** (contenedores con nombre único global).

```
# Pseudocódigo: subir un CSV a GCS
# from google.cloud import storage
#
# cliente = storage.Client()
# bucket = cliente.get_bucket("techstore-datos")
# blob = bucket.blob("ventas/ventas_2024.csv")
# blob.upload_from_filename("ventas_2024.csv")
```

```
# print(f"Archivo subido a gs://techstore-datos/ventas/ventas_2024.csv")
```

Los buckets tienen URLs tipo `gs://techstore-datos/ventas/ventas_2024.csv`` (gs = Google Storage).

Regiones y multi-regiones

GCP organiza sus datacenters en:

- **Regiones:** ``us-central1``, ``europe-west1``, ``southamerica-east1``
- **Multi-regiones:** ``US``, ``EU`` (abarcan varios datacenters de un continente)
- **Dual-region:** dos regiones específicas para alta disponibilidad

Elegir región:

- Más cercana a tus usuarios menor latencia
- Más cercana a ti menor coste
- Considera requisitos legales de residencia de datos

Costes en la nube: Compute vs Storage

En BigQuery (y muchos servicios cloud) pagas por separado:

- **Storage:** \$ por GB/mes por datos almacenados
- **Compute:** \$ por TB procesado por cada consulta

Esto cambia tu mentalidad:

- En local: preocuparte por RAM y CPU
- En BigQuery: preocuparte por cuántos GB escanea cada SELECT
- **Siempre usa `SELECT \ `project.dataset.table\ `.columnas`, no `SELECT \`** (esto escanea menos datos)

Ejercicios

1. Busca tres ventajas de usar BigQuery frente a SQLite para un dataset de 10 millones de filas
2. Investiga cuánto cuesta almacenar 1 GB en BigQuery (región US) y cuánto cuesta consultar 1 TB
3. ¿Qué modelo de servicio (IaaS, PaaS, SaaS) es Cloud Storage? ¿Y Looker Studio?
4. Explica con tus palabras qué significa "pago por uso" en cloud computing
5. ¿Por qué `SELECT *` puede ser peligroso en un data warehouse? ¿Y en SQLite?
6. Crea una tabla comparativa: SQLite vs BigQuery (5 criterios)
7. ¿Qué es un bucket? ¿Para qué sirve en el contexto de data analytics?
8. Investiga qué es "egress" en cloud y por qué puede encarecer tu factura
9. ¿Cuándo NO deberías migrar una base de datos a la nube?
10. Busca qué es el free tier de Google Cloud y qué incluye BigQuery

Capítulo 7: Google Cloud Platform — Configuración y Autenticación

Crea tu cuenta de GCP

Para seguir este libro necesitas una cuenta de Google Cloud Platform. No te preocupes: BigQuery tiene un **free tier** generoso que cubre 10 GB de almacenamiento y 1 TB de consultas por mes.

Paso 1: Crear cuenta

1. Ve a <https://console.cloud.google.com>
2. Inicia sesión con tu cuenta de Google (o crea una nueva)
3. Acepta los términos del servicio
4. Proporciona datos de facturación (te piden tarjeta, pero no te cobrarán mientras uses free tier)
5. Completa el registro

> **Importante:** GCP te da \$300 en créditos gratis por 90 días al crear tu primera cuenta. Puedes usar esto para experimentar sin riesgo.

Paso 2: Crear un proyecto

Un proyecto en GCP es el contenedor lógico que agrupa tus recursos, facturación y permisos.

1. En la consola, haz clic en el selector de proyectos (arriba a la izquierda)
2. Haz clic en "Nuevo proyecto"
3. Nombre: "techstore-analytics"
4. Ubicación: déjala por defecto (sin organización)
5. Haz clic en "Crear"

Anota el **Project ID** (suele ser `techstore-analytics-XXXXX`). Lo usarás en cada consulta.

Paso 3: Habilitar APIs

Cada servicio en GCP requiere habilitar su API:

1. Ve a "APIs y servicios" → "Biblioteca"
2. Busca y habilita:
 - BigQuery API
 - Cloud Storage API
3. Espera 1-2 minutos a que se activen

Autenticación: Service Accounts

Tu ordenador necesita identificarse ante Google Cloud para usar sus servicios. Hay dos formas principales:

Método	Cuándo usarlo
gcloud CLI	Desarrollo local, pruebas rápidas
Service Account (JSON key)	Código producción, automatización, scripts

Usaremos **gcloud CLI** para desarrollo y **Service Account** para los scripts Python.

Opción A: gcloud CLI (recomendada para desarrollo)

Instala Google Cloud SDK:

```
# Windows: descarga de https://cloud.google.com/sdk/docs/install
# O usa winget:
winget install Google.CloudSDK
```

Luego autentícate:

```
gcloud auth login
# Se abre tu navegador para autorizar
```

Configura tu proyecto activo:

```
gcloud config set project techstore-analytics-XXXXX
```

Verifica:

```
gcloud config list
gcloud auth list
```

Opción B: Service Account (para código Python)

1. En la consola: IAM y Admin Service Accounts
2. Haz clic en "Crear service account"
3. Nombre: `techstore-python`
4. Roles: BigQuery BigQuery Admin, Storage Storage Admin
5. Haz clic en "Hecho"
6. Selecciona la cuenta, ve a "Claves" "Agregar clave" JSON
7. Se descarga un archivo .json con las credenciales

Guarda este archivo de forma segura (nunca lo subas a GitHub). Lo cargarás en Python así:

```
from google.cloud import bigquery
from google.oauth2 import service_account

credentials = service_account.Credentials.from_service_account_file(
    "ruta/a/tu-archivo-credenciales.json"
)
cliente = bigquery.Client(credentials=credentials, project="techstore-analytics-XXXXX")
```

IAM — Quién puede hacer qué

IAM (Identity and Access Management) es el sistema de permisos de GCP. Sus tres componentes:

- **Quién:** cuenta de Google, service account, grupo
- **Rol:** conjunto de permisos (BigQuery Admin, BigQuery Data Viewer, etc.)
- **Recurso:** proyecto, dataset, tabla

Principio de **mínimo privilegio**: asigna solo los roles necesarios.

Rol BigQuery	Permisos
`bigquery.user`	Ejecutar consultas, crear datasets
`bigquery.dataViewer`	Leer datos y metadatos
`bigquery.dataEditor`	Crear/modificar tablas
`bigquery.admin`	Control total
`bigquery.jobUser`	Ejecutar trabajos (consultas/cargas)

Para este libro, asigna **BigQuery Admin** (es tu proyecto personal; en producción serías más restrictivo).

gcloud CLI — Comandos esenciales

```
# Ver proyectos disponibles
gcloud projects list

# Ver datasets de BigQuery
bq ls

# Ejecutar consulta SQL directa desde terminal
bq query "SELECT 1 AS numero"

# Ver información de una tabla
bq show techstore:ventas

# Subir archivo a Cloud Storage
gcloud storage cp ventas.csv gs://techstore-datos/
```

gcloud auth application-default

Para que tu código Python funcione sin cargar credenciales explícitamente:

```
gcloud auth application-default login
```

Esto crea credenciales por defecto que bibliotecas como `google-cloud-bigquery` usarán automáticamente.

```
from google.cloud import bigquery

# Sin credenciales explícitas – usa ADCs (Application Default Credentials)
cliente = bigquery.Client()
```

Ejercicios

1. Crea un proyecto en GCP llamado "techstore-analytics" y escribe aquí el Project ID

2. ¿Qué diferencia hay entre ``gcloud auth login`` y ``gcloud auth application-default login``?
3. ¿Para qué sirve una service account? ¿Cuándo usarías una vs tu cuenta personal?
4. Habilita la API de BigQuery y la de Cloud Storage en tu proyecto
5. Instala gcloud CLI (o verifica que ya lo tienes con ``gcloud --version``)
6. Enumera 5 roles de IAM en BigQuery y explica cada uno brevemente
7. ¿Qué significa "mínimo privilegio" en seguridad cloud?
8. Auténticate con ``gcloud auth login`` y configura tu proyecto activo
9. Explica por qué nunca debes subir un archivo JSON de service account a un repositorio público
10. Ejecuta ``bq query "SELECT CURRENT_DATE() AS hoy"`` desde tu terminal y pega el resultado

Capítulo 8: BigQuery — Tu Primer Data Warehouse en la Nube

¿Qué es BigQuery?

BigQuery es un **data warehouse** serverless y altamente escalable de Google Cloud. "Serverless" significa que no hay servidores que gestionar: Google aprovisiona, escala y mantiene la infraestructura por ti.

Características clave:

- **SQL estándar:** consultas con dialecto ANSI 2011 plus funciones propias
- **Escalado automático:** desde 1 fila hasta petabytes
- **Pago por uso:** pagas por almacenamiento + consultas ejecutadas
- **Streaming:** puedes insertar datos en tiempo real
- **Separación storage/compute:** almacenar es barato, consultar cuesta según datos escaneados

Arquitectura de BigQuery

```
Proyecto (techstore-analytics)
├─ Dataset (techstore)
│   ├── Tabla: customers
│   ├── Tabla: products
│   ├── Tabla: orders
│   ├── Tabla: order_items
│   └─ Tabla: employees
```

- **Dataset:** contenedor de tablas (similar a una base de datos SQLite)
- **Tabla:** filas y columnas, como en SQL (pero almacenada en columnar format — Capacitor/ColumnIO)
- **Job:** cada consulta, carga o exportación que ejecutas

Tu primer dataset en BigQuery

Desde la consola web:

1. Ve a BigQuery → console.cloud.google.com/bigquery

2. En el explorador, haz clic en los tres puntos junto a tu proyecto
3. "Crear dataset"
4. Dataset ID: techstore
5. Región: us-central1 (o la más cercana a ti)
6. "Crear dataset"

Desde CLI:

```
bq mk --dataset --location=us-central1 techstore-analytics:techstore
```

Tu primera tabla

Puedes crear tablas de varias formas:

Opción 1: Desde CSV en Cloud Storage

```
-- Esta es una consulta de carga (LOAD), no SELECT
-- Se ejecuta desde consola o bq CLI
LOAD DATA INTO techstore.customers
FROM FILES (
  format = 'CSV',
  uris = ['gs://techstore-datos/customers.csv']
)
OPTIONS (
  skip_leading_rows = 1,
  field_delimiter = ','
);
```

Opción 2: Desde consulta SQL

```
CREATE TABLE techstore.ventas_resumen AS
SELECT
  categoria,
  COUNT(*) AS total_ventas,
  ROUND(SUM(total), 2) AS ingresos_totales
FROM techstore.orders
GROUP BY categoria;
```

Opción 3: Crear tabla vacía con esquema

```
CREATE TABLE techstore.employees (
  employee_id INT64 NOT NULL,
  name STRING(100),
  position STRING(50),
  hire_date DATE,
  salary FLOAT64
);
```

Tipos de datos en BigQuery

Tipo BigQuery	Equivalente SQLite	Notas
---------------	--------------------	-------

<code>`INT64`</code>	<code>`INTEGER`</code>	Entero de 64 bits
<code>`FLOAT64`</code>	<code>`REAL`</code>	Coma flotante
<code>`STRING`</code>	<code>`TEXT`</code>	Unicode
<code>`BOOL`</code>	<code>`INTEGER` (0/1)</code>	Booleano
<code>`DATE`</code>	<code>`TEXT` (formato)</code>	Fecha sin hora
<code>`DATETIME`</code>	<code>`TEXT` (formato)</code>	Fecha + hora
<code>`TIMESTAMP`</code>	–	Momento exacto (con zona horaria)
<code>`NUMERIC`</code>	–	Decimal exacto de alta precisión
<code>`BYTES`</code>	<code>`BLOB`</code>	Datos binarios
<code>`ARRAY<TIPO>`</code>	–	Array repetido
<code>`STRUCT<...>`</code>	–	Registro anidado

SQL en BigQuery: diferencias con SQLite

```
-- SQLite
SELECT datetime('now');

-- BigQuery
SELECT CURRENT_DATETIME();
SELECT CURRENT_TIMESTAMP();

-- SQLite
SELECT * FROM orders LIMIT 10;

-- BigQuery (igual)
SELECT * FROM orders LIMIT 10;

-- BigQuery: nombre completo con proyecto.dataset.tabla
SELECT * FROM techstore-analytics.techstore.orders LIMIT 10;

-- 0 con backticks
SELECT * FROM `techstore-analytics.techstore.orders` LIMIT 10;

-- Puedes omitir proyecto y dataset si es el activo
SELECT * FROM orders LIMIT 10;
```

Funciones útiles de BigQuery

```
-- Fechas
SELECT
  EXTRACT(YEAR FROM order_date) AS anio,
  EXTRACT(MONTH FROM order_date) AS mes,
  DATE_TRUNC(order_date, MONTH) AS mes_inicio,
  FORMAT_DATE('%Y-%m', order_date) AS anio_mes
FROM orders;

-- Strings
SELECT
  UPPER(name) AS nombre_mayus,
```

```

    LOWER(email) AS email_minus,
    CONCAT(first_name, ' ', last_name) AS nombre_completo,
    SPLIT(email, '@')[OFFSET(0)] AS usuario,
    LENGTH(name) AS longitud
FROM customers;

-- NULLs
SELECT
    COALESCE(phone, 'NO TIENE') AS telefono,
    IFNULL(email, 'sin_email@email.com') AS email
FROM customers;

-- Ventanas
SELECT
    order_id, customer_id, total,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY total DESC) AS rn,
    RANK() OVER (ORDER BY total DESC) AS ranking_global,
    SUM(total) OVER (PARTITION BY customer_id) AS total_cliente
FROM orders;

```

Consultas prácticas en BigQuery

```

-- ¿Cuánto almacenamiento usa cada tabla?
SELECT
    table_name,
    ROUND(size_bytes / POW(1024, 3), 2) AS size_gb,
    row_count
FROM `techstore.INFORMATION_SCHEMA.TABLES`;

-- Consulta que escanea menos datos
SELECT
    DATE_TRUNC(order_date, MONTH) AS mes,
    COUNT(*) AS pedidos
FROM orders
WHERE order_date >= '2024-01-01' -- Filtro por partición
GROUP BY mes;

-- JOINS funcionan igual que en SQLite
SELECT
    c.name,
    COUNT(o.order_id) AS num_pedidos,
    ROUND(SUM(o.total), 2) AS gasto_total
FROM orders AS o
JOIN customers AS c USING (customer_id)
GROUP BY c.name
ORDER BY gasto_total DESC;

```

Estimación de costes antes de ejecutar

BigQuery te dice cuántos GB escaneará tu consulta **antes de ejecutarla**:

```
# CLI: --dry_run estima el coste sin ejecutar
bq query --dry_run "SELECT * FROM techstore.orders"

# Consola: aparece "Bytes que se leerán" arriba del botón Ejecutar
# Python: job_config.dry_run = True
```

> **Regla de oro:** 1 TB escaneado $\approx$ \$5 USD (a precio estándar). Siempre prefiltra y selecciona solo columnas necesarias.

Ejercicios

1. Crea el dataset `techstore` en BigQuery desde la consola web
2. ¿Cuál es la diferencia entre un dataset en BigQuery y una base de datos en SQLite?
3. Ejecuta `SELECT CURRENT\_DATE() AS hoy, CURRENT\_TIMESTAMP() AS ahora` en BigQuery
4. ¿Qué significa que BigQuery sea "serverless"? Nombra dos ventajas
5. Crea una tabla llamada `test` con columnas `id INT64, nombre STRING, fecha DATE`
6. Inserta 3 registros en `test` usando INSERT INTO
7. Elimina la tabla `test` con DROP TABLE
8. ¿Para qué sirve `SELECT ... FROM tabla WHERE ... LIMIT o`? (pista: estimación de costes)
9. Usa INFORMATION_SCHEMA para listar todas las tablas de tu dataset techstore
10. Explica qué función cumple `DATE\_TRUNC` y escribe un ejemplo

Capítulo 9: BigQuery Avanzado — Particiones, Clustering y Optimización

El problema del full scan

Cuando ejecutas ``SELECT FROM orders``, BigQuery escanea todas las filas de la tabla*, aunque solo necesites un mes de datos. Si la tabla tiene 10 TB, pagas por escanear 10 TB cada vez.

BigQuery ofrece dos mecanismos para evitar esto: **particiones** y **clustering**.

Particionamiento (Partitioning)

Una tabla particionada divide los datos en segmentos basados en una columna (normalmente fecha). Al consultar con un filtro en esa columna, BigQuery solo lee las particiones relevantes.

Tipos de particionamiento

Tipo	Columna	Ejemplo
Por tiempo (ingesta)	<code>`_PARTITIONTIME`</code>	Datos del día X
Por columna DATE/DATETIME	Una columna date	<code>`order_date`</code>
Por entero (rango)	Columna INT64	<code>`customer_id` RANGE BETWEEN 1 AND 100000</code>

Crear tabla particionada

```
-- Particionada por order_date (costo mensual)
CREATE TABLE techstore.orders_partitioned
PARTITION BY DATE_TRUNC(order_date, MONTH)
AS
SELECT * FROM techstore.orders;

-- Particionada por día (más granular)
CREATE TABLE techstore.orders_daily
PARTITION BY order_date
OPTIONS (
  partition_expiration_days = 365 -- Borra particiones > 1 año
) AS
SELECT * FROM techstore.orders;

-- Con entero (por rango de customer_id)
CREATE TABLE techstore.customers_partitioned
PARTITION BY RANGE_BUCKET(customer_id, GENERATE_ARRAY(0, 250001, 50000))
```

```
AS
SELECT * FROM techstore.customers;
```

Consultas en tablas particionadas

```
-- BigQuery solo escanea las particiones de 2024
SELECT
  DATE_TRUNC(order_date, MONTH) AS mes,
  COUNT(*) AS pedidos,
  ROUND(SUM(total), 2) AS ingresos
FROM techstore.orders_partitioned
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY mes;

-- Pseudocolumna _PARTITIONTIME (para particiones por ingesta)
SELECT DISTINCT _PARTITIONTIME FROM techstore.orders_partitioned;
```

> **Beneficio:** si tienes 5 años de datos pero filtras por 1 mes, BigQuery escanea solo ese mes (~1/60 de los datos). **Ahorro: ~98% del coste.**

Clustering

Clustering ordena físicamente los datos dentro de cada partición según una o más columnas. No reduce el escaneo como las particiones, pero acelera las consultas con filtros en esas columnas.

Crear tabla con clustering

```
CREATE TABLE techstore.orders_clustered
PARTITION BY DATE_TRUNC(order_date, MONTH)
CLUSTER BY customer_id, category
AS
SELECT * FROM techstore.orders;
```

¿Cuándo usar clustering?

Situación	Recomendación
Filtros frecuentes por columna X	Clustering por X
Columnas con alta cardinalidad (muchos valores únicos)	Buen candidato a clustering
Columnas con baja cardinalidad (pocos valores, ej: categorías)	Buen candidato a clustering
Tablas < 1 GB	No vale la pena (overhead)
Consultas sin filtro	No beneficia

Límite: hasta 4 columnas de clustering. El orden importa: pon primero la columna más usada en filtros.

Clustering con múltiples columnas

```
CREATE TABLE techstore.orders_clustered_v2
PARTITION BY DATE_TRUNC(order_date, MONTH)
CLUSTER BY category, customer_id, status
AS
SELECT * FROM techstore.orders;
```

En este caso, las consultas que filtren por `category` serán más rápidas. Las que filtren por `status` sin `category` se beneficiarán menos.

Buenas prácticas de optimización

1. Selecciona solo las columnas necesarias

```
-- MAL: escanea todas las columnas
SELECT * FROM orders;

-- BIEN: escanea solo 3 columnas
SELECT order_date, customer_id, total FROM orders;
```

2. Prefiltra antes de JOIN

```
-- MAL: JOIN primero, filtro después
SELECT c.name, o.total
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '2024-01-01';

-- BIEN: filtra primero, JOIN después (BigQuery lo optimiza, pero es buena práctica)
-- Usa subquery o CTE para filtrar datos antes del JOIN
WITH orders_2024 AS (
    SELECT * FROM orders WHERE order_date >= '2024-01-01'
)
SELECT c.name, o.total
FROM customers c
JOIN orders_2024 o ON c.customer_id = o.customer_id;
```

3. Usa LIMIT o para estimar coste

```
-- Muestra el tamaño de datos a escanear sin ejecutar realmente
SELECT * FROM techstore.orders LIMIT 0;
```

En la consola verás "Bytes que se leerán: 1.2 GB" — eso es lo que costaría un `SELECT \*`.

4. Evita SELECT DISTINCT en columnas grandes

```
-- En lugar de DISTINCT sobre columna larga
SELECT DISTINCT description FROM products;

-- Considera agregar una tabla de dimensiones
-- o usar GROUP BY si necesitas agregaciones
```

5. Usa aproximaciones cuando sea aceptable

```
-- Exacto (lento en tablas enormes)
SELECT COUNT(DISTINCT customer_id) FROM orders;

-- Aproximado (rápido, margen de error ~1%)
SELECT APPROX_COUNT_DISTINCT(customer_id) FROM orders;
```

6. Materializa resultados intermedios

```
-- Si usas la misma subconsulta múltiples veces, créala como tabla
CREATE TABLE techstore.ventas_2024 AS
SELECT * FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';
```

Slot-based architecture

BigQuery ejecuta consultas usando **slots** (unidades de procesamiento). Cada slot equivale aproximadamente a un core de CPU con memoria.

- **On-demand:** hasta 2000 slots simultáneos (compartidos con otros usuarios)
- **Flat-rate:** compras slots fijos (rendimiento predecible, mejor para producción)

La mayoría de usuarios empezamos con **on-demand**. Si tus consultas se vuelven lentas, puedes considerar flat-rate.

INFORMATION_SCHEMA para monitorización

```
-- Consultas ejecutadas hoy
SELECT
  query,
  ROUND(total_bytes_processed / POW(1024, 3), 2) AS gb_procesados,
  total_slot_ms / 1000 AS segundos_slot,
  TIMESTAMP_DIFF(end_time, start_time, SECOND) AS duracion_seg
FROM `region-us.INFORMATION_SCHEMA.JOBS`
WHERE EXTRACT(DATE FROM creation_time) = CURRENT_DATE()
  AND job_type = 'QUERY'
ORDER BY total_bytes_processed DESC;
```

Ejercicios

1. Crea una tabla particionada por mes con los datos de orders
2. Explica con tus palabras la diferencia entre partitioning y clustering
3. ¿Cuántas columnas de clustering puedes definir como máximo?
4. Crea una tabla clustered por `category` y `customer\_id`
5. Ejecuta una consulta con y sin partición: ¿cuántos GB escanea cada una?
6. ¿Qué es un slot en BigQuery? ¿Para qué sirve?
7. Usa INFORMATION_SCHEMA.JOBS para listar las 3 consultas más caras de hoy
8. ¿Por qué SELECT * es una mala práctica en BigQuery? ¿En qué casos se justifica?
9. ¿Cuándo NO usarías particionamiento? Plantea un ejemplo
10. Crea una tabla con `partition\_expiration\_days = 90` y explica qué hace ese parámetro

Capítulo 10: Python + BigQuery — Consultas y Automatización

La biblioteca google-cloud-bigquery

Python se conecta a BigQuery mediante la biblioteca oficial `google-cloud-bigquery`.

```
pip install google-cloud-bigquery google-cloud-storage pandas db-dtypes
```

- `google-cloud-bigquery`: cliente para consultas y gestión
- `google-cloud-storage`: para interactuar con Cloud Storage
- `pandas`: para convertir resultados a DataFrames
- `db-dtypes`: tipos de datos adicionales para compatibilidad pandas

Conectar y consultar

```
from google.cloud import bigquery

# Cliente usando Application Default Credentials
cliente = bigquery.Client()

# Consulta simple
query = """
    SELECT
        DATE_TRUNC(order_date, MONTH) AS mes,
        COUNT(*) AS pedidos,
        ROUND(SUM(total), 2) AS ingresos
    FROM techstore.orders
    WHERE order_date >= '2024-01-01'
    GROUP BY mes
    ORDER BY mes
"""

# Ejecutar y obtener resultados como lista de filas
rows = cliente.query(query).result()
for row in rows:
    print(f"{row.mes}: {row.pedidos} pedidos, ${row.ingresos}")
```

Resultados como DataFrame de pandas


```
import pandas as pd

query = """
    SELECT c.name, c.city, COUNT(o.order_id) AS pedidos
    FROM techstore.orders AS o
    JOIN techstore.customers AS c USING (customer_id)
    GROUP BY c.name, c.city
    ORDER BY pedidos DESC
    LIMIT 20
    """

df = cliente.query(query).to_dataframe()
print(df.head())
print(f"Shape: {df.shape}")
```

Parámetros en consultas (SQL injection safe)

```
from google.cloud.bigquery import QueryJobConfig, ScalarQueryParameter

query = """
    SELECT order_date, customer_id, total
    FROM techstore.orders
    WHERE order_date BETWEEN @fecha_inicio AND @fecha_fin
    ORDER BY order_date
    """

job_config = QueryJobConfig(
    query_parameters=[
        ScalarQueryParameter("fecha_inicio", "DATE", "2024-01-01"),
        ScalarQueryParameter("fecha_fin", "DATE", "2024-06-30"),
    ]
)

df = cliente.query(query, job_config=job_config).to_dataframe()
```

Dry run (estimar coste sin ejecutar)

```
from google.cloud.bigquery import QueryJobConfig

job_config = QueryJobConfig(dry_run=True)

query = "SELECT * FROM techstore.orders"
job = cliente.query(query, job_config=job_config)

print(f"Bytes a escanear: {job.total_bytes_processed:,}")
print(f"Costo estimado: ${job.total_bytes_processed / 1e12 * 5:.4f} USD")
```

Cargar datos desde pandas a BigQuery

```
import pandas as pd
```

```

# Suponiendo que ya tienes un DataFrame con los datos
df = pd.read_csv("codigos/techstore_export/orders.csv")

# Configuración de carga
job_config = bigquery.LoadJobConfig(
    write_disposition="WRITE_TRUNCATE", # Reemplaza la tabla si existe
    autodetect=True, # Detecta esquema automáticamente
)

# Cargar DataFrame a BigQuery
tabla_id = "techstore.orders_from_pandas"
job = cliente.load_table_from_dataframe(df, tabla_id, job_config=job_config)
job.result() # Espera a que termine

print(f"Cargadas {job.output_rows} filas en {tabla_id}")

```

Exportar datos de BigQuery a local

```

# Consultar y guardar como CSV local
df = cliente.query("""
    SELECT * FROM techstore.orders
    WHERE order_date >= '2024-01-01'
""").to_dataframe()

df.to_csv("codigos/orders_2024_export.csv", index=False)
print(f"Exportadas {len(df)} filas")

# También puedes exportar directamente a GCS
# Exportación directa desde SQL (más eficiente para datasets grandes)

```

Gestión de tablas desde Python

```

# Listar datasets
datasets = list(cliente.list_datasets())
for ds in datasets:
    print(f"Dataset: {ds.dataset_id}")

# Listar tablas de un dataset
dataset_ref = cliente.dataset("techstore")
tables = list(cliente.list_tables(dataset_ref))
for table in tables:
    print(f"Tabla: {table.table_id}")

# Obtener metadatos de una tabla
table = cliente.get_table("techstore.orders")
print(f"Filas: {table.num_rows}")
print(f"Tamaño: {table.num_bytes / 1e9:.2f} GB")
print(f"Esquema:")
for field in table.schema:

```

```

        print(f" - {field.name}: {field.field_type}")

# Crear tabla desde Python
schema = [
    bigquery.SchemaField("log_id", "INT64", mode="REQUIRED"),
    bigquery.SchemaField("accion", "STRING", mode="REQUIRED"),
    bigquery.SchemaField("fecha", "DATETIME", mode="REQUIRED"),
    bigquery.SchemaField("usuario", "STRING"),
]
table = bigquery.Table("techstore.audit_logs", schema=schema)
table = cliente.create_table(table)
print(f"Tabla creada: {table.table_id}")

```

Script completo: migración de TechStore a BigQuery

```

"""
migrar_techstore_a_bigquery.py
Migra la base de datos SQLite de TechStore a BigQuery.

Uso:
    python migrar_techstore_a_bigquery.py
"""

import sqlite3
import pandas as pd
from google.cloud import bigquery

# --- Configuración ---
SQLITE_PATH = "../codigos/techstore.db"
PROJECT = "techstore-analytics" # Cambia por tu Project ID
DATASET = "techstore"

TABLAS = [
    "customers",
    "products",
    "orders",
    "order_items",
    "employees",
]

cliente = bigquery.Client(project=PROJECT)

# --- Crear dataset si no existe ---
dataset_ref = cliente.dataset(DATASET)
try:
    cliente.get_dataset(dataset_ref)
    print(f"Dataset {DATASET} ya existe")
except Exception:
    dataset = bigquery.Dataset(dataset_ref)
    dataset.location = "us-central1"
    cliente.create_dataset(dataset)

```

```

print(f"Dataset {DATASET} creado")

# --- Migrar cada tabla ---
conn = sqlite3.connect(SQLITE_PATH)

for tabla in TABLAS:
    print(f"Migrando {tabla}...")

    # Leer desde SQLite
    df = pd.read_sql_query(f"SELECT * FROM {tabla}", conn)
    print(f"  Filas leídas: {len(df)}")

    # Cargar a BigQuery
    tabla_id = f"{DATASET}.{tabla}"
    job_config = bigquery.LoadJobConfig(
        write_disposition="WRITE_TRUNCATE",
        autodetect=True,
    )

    job = cliente.load_table_from_dataframe(df, tabla_id, job_config=job_config)
    job.result()

    # Verificar
    table = cliente.get_table(tabla_id)
    print(f"  Cargadas: {table.num_rows} filas, {table.num_bytes / 1e6:.2f} MB")

conn.close()
print("Migración completada.")

```

Buenas prácticas con Python + BigQuery

1. **Usa DataFrame para análisis exploratorio:** convierte resultados a pandas, explora, y luego escribe la query optimizada
2. **Siempre usa dry_run** antes de consultas grandes
3. **Usa parámetros** en vez de f-strings para evitar SQL injection
4. **Configura timeout** en consultas largas: ``result(timeout=30)``
5. **Agrupar cargas:** mejor cargas grandes y pocas que muchas pequeñas
6. **Usa WRITE_TRUNCATE con cuidado:** borra datos existentes; usa WRITE_APPEND para añadir

Ejercicios

1. Instala google-cloud-bigquery y pandas en tu entorno
2. Conecta Python a BigQuery y lista los datasets disponibles
3. Ejecuta una consulta que devuelva las 5 categorías más vendidas como DataFrame
4. Usa dry_run para estimar cuánto costaría un `SELECT * FROM orders`
5. Carga el DataFrame de orders desde SQLite a BigQuery desde Python
6. Escribe una función que reciba una fecha y devuelva las ventas de ese mes

7. Exporta los resultados de una consulta a un archivo CSV local
8. ¿Qué hace `write_disposition="WRITE_APPEND"`? ¿Cuándo lo usarías?
9. Crea una nueva tabla desde Python llamada `daily_summary` con 3 columnas
10. Escribe un script que compare el número de filas de cada tabla entre SQLite y BigQuery

Checkpoint 2: Migrar TechStore a BigQuery

Objetivo

Has aprendido los fundamentos de cloud computing, BigQuery, particionamiento, clustering y cómo conectar Python con BigQuery. Ahora es momento de aplicar todo: **migrar la base de datos TechStore desde SQLite a BigQuery** y validar que los datos sean correctos.

Requisitos

- Cuenta de Google Cloud con proyecto creado
- BigQuery API habilitada
- gcloud CLI instalado y autenticado
- Python con google-cloud-bigquery instalado
- Archivo `techstore.db` en `codigos/`

Paso 1: Preparar el dataset en BigQuery

Crea el dataset `techstore` en tu proyecto de GCP:

```
bq mk --dataset --location=us-central1 PROJECT_ID:techstore
```

Paso 2: Ejecutar el script de migración

Crea el archivo `codigos/migrar\_techstore\_a\_bigquery.py` con el siguiente contenido:

```
"""
migrar_techstore_a_bigquery.py
Migra la base de datos SQLite de TechStore a BigQuery.
"""

import sqlite3
import pandas as pd
from google.cloud import bigquery

# --- Configuración ---
SQLITE_PATH = "techstore.db"
PROJECT = "techstore-analytics" # CAMBIA por tu Project ID
```

```

DATASET = "techstore"

TABLAS = [
    "customers",
    "products",
    "orders",
    "order_items",
    "employees",
]

cliente = bigquery.Client(project=PROJECT)

# --- Crear dataset si no existe ---
dataset_ref = cliente.dataset(DATASET)
try:
    cliente.get_dataset(dataset_ref)
    print(f"Dataset {DATASET} ya existe")
except Exception:
    dataset = bigquery.Dataset(dataset_ref)
    dataset.location = "us-central1"
    cliente.create_dataset(dataset)
    print(f"Dataset {DATASET} creado")

# --- Migrar cada tabla ---
conn = sqlite3.connect(SQLITE_PATH)

for tabla in TABLAS:
    print(f"Migrando {tabla}...")

    # Leer desde SQLite
    df = pd.read_sql_query(f"SELECT * FROM {tabla}", conn)
    print(f"  Filas leídas: {len(df)}")

    # Cargar a BigQuery
    tabla_id = f"{DATASET}.{tabla}"
    job_config = bigquery.LoadJobConfig(
        write_disposition="WRITE_TRUNCATE",
        autodetect=True,
    )

    job = cliente.load_table_from_dataframe(df, tabla_id, job_config=job_config)
    job.result()

    # Verificar
    table = cliente.get_table(tabla_id)
    print(f"  Cargadas: {table.num_rows} filas, {table.num_bytes / 1e6:.2f} MB")

conn.close()
print("Migración completada.")

```

Ejecútalo:

```
python codigos/migrar_techstore_a_bigquery.py
```

Paso 3: Validar la migración

Ejecuta estas consultas en la consola de BigQuery o desde Python para verificar que los datos sean correctos:

```
-- Verificar número de filas por tabla
SELECT 'customers' AS tabla, COUNT(*) AS filas FROM techstore.customers
UNION ALL
SELECT 'products', COUNT(*) FROM techstore.products
UNION ALL
SELECT 'orders', COUNT(*) FROM techstore.orders
UNION ALL
SELECT 'order_items', COUNT(*) FROM techstore.order_items
UNION ALL
SELECT 'employees', COUNT(*) FROM techstore.employees;
```

```
-- Comparar ingresos totales con el EDA del Libro 1
SELECT
  ROUND(SUM(total), 2) AS ingresos_totales,
  COUNT(*) AS total_pedidos,
  ROUND(AVG(total), 2) AS ticket_promedio
FROM techstore.orders;
```

```
-- Top 10 clientes por gasto
SELECT
  c.name,
  c.city,
  COUNT(o.order_id) AS pedidos,
  ROUND(SUM(o.total), 2) AS gasto_total
FROM techstore.orders AS o
JOIN techstore.customers AS c ON o.customer_id = c.customer_id
GROUP BY c.name, c.city
ORDER BY gasto_total DESC
LIMIT 10;
```

```
-- Ventas por mes
SELECT
  DATE_TRUNC(order_date, MONTH) AS mes,
  COUNT(*) AS pedidos,
  ROUND(SUM(total), 2) AS ingresos
FROM techstore.orders
GROUP BY mes
ORDER BY mes;
```

Paso 4: Crear tabla particionada

A partir de los datos migrados, crea una versión optimizada:

```
CREATE TABLE techstore.orders_partitioned
PARTITION BY DATE_TRUNC(order_date, MONTH)
CLUSTER BY customer_id
AS
```



```
SELECT * FROM techstore.orders;
```

Verifica la reducción de coste:

```
-- En la tabla original (sin particionar)
SELECT COUNT(*) FROM techstore.orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31';

-- En la tabla particionada
SELECT COUNT(*) FROM techstore.orders_partitioned
WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31';
```

Compara los "Bytes que se leerán" en la consola antes de ejecutar.

Paso 5: Automatizar desde Python

Crea `codigos/verificar\_migracion.py`:

```
"""
verificar_migracion.py
Verifica que la migración a BigQuery sea correcta.
Compara totales entre SQLite y BigQuery.
"""

import sqlite3
from google.cloud import bigquery

SQLITE_PATH = "techstore.db"
PROJECT = "techstore-analytics" # CAMBIA
DATASET = "techstore"

conexion_bq = bigquery.Client(project=PROJECT)
conexion_sqlite = sqlite3.connect(SQLITE_PATH)

TABLAS = ["customers", "products", "orders", "order_items", "employees"]

print("Comparación de filas por tabla:")
print(f"{'Tabla':<15} {'SQLite':<10} {'BigQuery':<10} {'Coinciden':<10}")
print("-" * 50)

for tabla in TABLAS:
    # SQLite
    cursor = conexion_sqlite.execute(f"SELECT COUNT(*) FROM {tabla}")
    count_sqlite = cursor.fetchone()[0]

    # BigQuery
    query = f"SELECT COUNT(*) AS cnt FROM {DATASET}.{tabla}"
    rows = conexion_bq.query(query).result()
    count_bq = list(rows)[0].cnt

    coincide = "" if count_sqlite == count_bq else ""
    print(f"{tabla:<15} {count_sqlite:<10} {count_bq:<10} {coincide:<10}")

conexion_sqlite.close()
```

```
print("\nVerificación completada.")
```

Entregables del Checkpoint 2

Al completar este checkpoint deberías tener:

- ☐ Dataset `techstore` creado en BigQuery
- ☐ 5 tablas migradas desde SQLite (customers, products, orders, order_items, employees)
- ☐ Script de migración funcionando (`migrar\_techstore\_a\_bigquery.py`)
- ☐ Script de verificación funcionando (`verificar\_migracion.py`)
- ☐ Tabla particionada `orders\_partitioned` creada
- ☐ Todas las consultas de validación ejecutadas y resultados documentados

Preguntas de reflexión

1. ¿Cuánto tiempo tomó la migración? ¿Fue más rápido de lo que esperabas?
2. ¿Qué diferencia de coste encontraste entre consultar la tabla original vs la particionada?
3. ¿Qué ventajas ves de tener TechStore en BigQuery vs SQLite para un equipo de 5 analistas?
4. Si tuvieras 50 millones de pedidos en lugar de 6000, ¿cómo cambiaría tu enfoque?

¡Felicidades! TechStore ya está en la nube. En el próximo proyecto conectarás estos datos a Looker Studio para crear dashboards profesionales.

Capítulo 11: BigQuery — SQL Analítico y Nested Fields

Más allá del SQL básico

En el Libro 1 dominaste SQLite con SELECT, JOIN, GROUP BY y funciones de ventana. BigQuery lleva el SQL analítico al siguiente nivel con tipos anidados, funciones de machine learning y capacidades geoespaciales.

En este capítulo explorarás las herramientas SQL que hacen de BigQuery un data warehouse analítico de clase mundial.

Nested fields (STRUCT)

BigQuery permite columnas de tipo **STRUCT** (registros anidados). En lugar de tener tablas separadas unidas por JOIN, puedes incluir sub-registros dentro de una fila.

```
-- Crear tabla con columna anidada
CREATE TABLE techstore.orders_nested AS
SELECT
  o.order_id,
  o.order_date,
  o.total,
  o.status,
  STRUCT(
    c.customer_id,
    c.name,
    c.email,
    c.city
  ) AS customer,
  STRUCT(
    e.employee_id,
    e.name AS employee_name,
    e.position
  ) AS seller
FROM techstore.orders AS o
JOIN techstore.customers AS c ON o.customer_id = c.customer_id
JOIN techstore.employees AS e ON o.employee_id = e.employee_id;
```

Consultar campos anidados

```
-- Acceder con punto (.)
SELECT
  order_id,
  customer.name AS cliente,
  customer.city AS ciudad,
  seller.employee_name AS vendedor
FROM techstore.orders_nested
LIMIT 10;
```

Arrays dentro de STRUCTs

```
-- Crear tabla con ARRAY<STRUCT>
CREATE TABLE techstore.customer_summary AS
SELECT
  customer_id,
  name,
  email,
  ARRAY_AGG(
    STRUCT(
      order_id,
      order_date,
      total,
      status
    )
  ) AS orders
FROM techstore.orders_nested
GROUP BY customer_id, name, email;

-- Desanidar (UNNEST)
SELECT
  cs.name,
  order_details.order_id,
  order_details.total
FROM techstore.customer_summary AS cs,
UNNEST(cs.orders) AS order_details
WHERE order_details.total > 1000;
```

Funciones analíticas avanzadas

Percentiles y distribución

```
-- Percentiles del ticket promedio
SELECT
  ROUND(AVG(total), 2) AS promedio,
  APPROX_QUANTILES(total, 4)[OFFSET(1)] AS p25,
  APPROX_QUANTILES(total, 4)[OFFSET(2)] AS p50_mediana,
  APPROX_QUANTILES(total, 4)[OFFSET(3)] AS p75,
  MAX(total) AS maximo
FROM techstore.orders;
```

Correlación

```
-- Correlación entre cantidad de productos y total del pedido
-- (necesitamos agregar por pedido)
WITH resumen AS (
    SELECT
        oi.order_id,
        SUM(oi.quantity) AS total_items,
        SUM(oi.quantity * oi.unit_price) AS total
    FROM techstore.order_items AS oi
    GROUP BY oi.order_id
)
SELECT
    CORR(total_items, total) AS correlacion_items_total
FROM resumen;
```

Cohortes (análisis temporal)

```
-- Análisis de cohortes por mes de primera compra
WITH primera_compra AS (
    SELECT
        customer_id,
        DATE_TRUNC(MIN(order_date), MONTH) AS cohorte_mes
    FROM techstore.orders
    GROUP BY customer_id
),
actividad AS (
    SELECT
        pc.customer_id,
        pc.cohorte_mes,
        DATE_TRUNC(o.order_date, MONTH) AS mes_actividad,
        DATE_DIFF(
            DATE_TRUNC(o.order_date, MONTH),
            pc.cohorte_mes,
            MONTH
        ) AS mes_cohorte
    FROM techstore.orders AS o
    JOIN primera_compra AS pc ON o.customer_id = pc.customer_id
)
SELECT
    cohorte_mes,
    mes_cohorte,
    COUNT(DISTINCT customer_id) AS clientes_activos
FROM actividad
GROUP BY cohorte_mes, mes_cohorte
ORDER BY cohorte_mes, mes_cohorte;
```

Window functions avanzadas

```
-- Diferencia mes a mes
WITH ventas_mensuales AS (
    SELECT
        DATE_TRUNC(order_date, MONTH) AS mes,
```

```

        ROUND(SUM(total), 2) AS ingresos
    FROM techstore.orders
    GROUP BY mes
)
SELECT
    mes,
    ingresos,
    LAG(ingresos) OVER (ORDER BY mes) AS mes_anterior,
    ROUND(ingresos - LAG(ingresos) OVER (ORDER BY mes), 2) AS variacion,
    ROUND(
        (ingresos - LAG(ingresos) OVER (ORDER BY mes))
        / LAG(ingresos) OVER (ORDER BY mes) * 100,
        2
    ) AS variacion_pct
FROM ventas_mensuales
ORDER BY mes;

```

```

-- Running total (acumulado)
WITH ventas_mensuales AS (
    SELECT
        DATE_TRUNC(order_date, MONTH) AS mes,
        ROUND(SUM(total), 2) AS ingresos
    FROM techstore.orders
    GROUP BY mes
)
SELECT
    mes,
    ingresos,
    ROUND(SUM(ingresos) OVER (ORDER BY mes), 2) AS acumulado,
    ROUND(AVG(ingresos) OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS media_movil_3m
FROM ventas_mensuales
ORDER BY mes;

```

Expresiones regulares en BigQuery

```

-- Extraer dominio del email
SELECT
    email,
    REGEXP_EXTRACT(email, r'@(.+)') AS dominio,
    REGEXP_CONTAINS(email, r'gmail|hotmail|yahoo') AS es_correo_popular
FROM techstore.customers;

-- Validar formato de teléfono
SELECT
    name,
    phone,
    CASE
        WHEN REGEXP_CONTAINS(phone, r'^\+\d{2,3}\s?\d{9,10}$') THEN 'Formato válido'
        ELSE 'Formato inválido'
    END AS valido
FROM techstore.customers;

```

Funciones de fecha avanzadas

```
-- Días entre pedidos por cliente
WITH pedidos_con_fechas AS (
  SELECT
    customer_id,
    order_date,
    LAG(order_date) OVER (
      PARTITION BY customer_id ORDER BY order_date
    ) AS fecha_anterior
  FROM techstore.orders
)
SELECT
  customer_id,
  order_date,
  fecha_anterior,
  DATE_DIFF(order_date, fecha_anterior, DAY) AS dias_entre_pedidos
FROM pedidos_con_fechas
WHERE fecha_anterior IS NOT NULL
ORDER BY dias_entre_pedidos DESC
LIMIT 10;
```

Scripting en BigQuery

BigQuery soporta scripts con variables, bucles y condicionales:

```
-- Variables
DECLARE mes_inicio DATE DEFAULT '2024-01-01';
DECLARE mes_fin DATE DEFAULT '2024-06-30';

SELECT
  DATE_TRUNC(order_date, MONTH) AS mes,
  COUNT(*) AS pedidos
FROM techstore.orders
WHERE order_date BETWEEN mes_inicio AND mes_fin
GROUP BY mes
ORDER BY mes;
```

```
-- Bucle para crear tablas por mes (ejemplo conceptual)
DECLARE i INT64 DEFAULT 1;
DECLARE mes STRING;

WHILE i <= 12 DO
  SET mes = FORMAT('%04d-%02d', 2024, i);
  EXECUTE IMMEDIATE FORMAT("""
    CREATE OR REPLACE TABLE techstore.ventas_%s AS
    SELECT * FROM techstore.orders
    WHERE FORMAT_DATE('%Y-%m', order_date) = '%s'
    """, mes, mes);
  SET i = i + 1;
END WHILE;
```

BQML: Machine Learning en SQL

BigQuery ML permite crear y ejecutar modelos de ML directamente con SQL:

```
-- Crear modelo de regresión lineal para predecir total del pedido
CREATE OR REPLACE MODEL techstore.modelo_ingresos
OPTIONS(model_type='linear_reg', input_label_cols=['total']) AS
SELECT
  o.total,
  EXTRACT(YEAR FROM o.order_date) AS anio,
  EXTRACT(MONTH FROM o.order_date) AS mes,
  EXTRACT(DAYOFWEEK FROM o.order_date) AS dia_semana,
  COUNT(DISTINCT oi.product_id) AS productos_distintos,
  SUM(oi.quantity) AS total_items
FROM techstore.orders AS o
JOIN techstore.order_items AS oi USING (order_id)
GROUP BY o.order_id, o.total, o.order_date;

-- Evaluar el modelo
SELECT * FROM ML.EVALUATE(MODEL techstore.modelo_ingresos);

-- Predecir
SELECT * FROM ML.PREDICT(
  MODEL techstore.modelo_ingresos,
  (
    SELECT
      1299.99 AS total,
      2025 AS anio,
      1 AS mes,
      2 AS dia_semana,
      3 AS productos_distintos,
      5 AS total_items
  )
);
```

Ejercicios

1. Crea una tabla con STRUCT que combine orders y customers
2. Escribe una consulta que use UNNEST para desanidar un array
3. Calcula los percentiles 10, 50 y 90 del total de pedidos
4. ¿Qué hace APPROX_QUANTILES y por qué es útil?
5. Crea una consulta que calcule la media móvil de 3 meses de ingresos
6. Usa REGEXP_EXTRACT para extraer el nombre de usuario (antes de @) de los emails
7. ¿Qué diferencia hay entre STRUCT y ARRAY en BigQuery?
8. Escribe un script con variable para filtrar pedidos por año
9. Investiga: ¿qué otros modelos de ML soporta BQML además de regresión lineal?
10. Crea un análisis de cohortes mensual para los clientes de TechStore

Capítulo 12: Data Pipelines con Python y BigQuery

¿Qué es un data pipeline?

Un **data pipeline** es un proceso automatizado que mueve datos desde un origen hasta un destino, aplicando transformaciones en el camino. El esquema clásico:

```
Origen → Extraer → Transformar → Cargar → Destino
```

En nuestro contexto:

- **Origen:** SQLite local, APIs, CSVs
- **Transformación:** pandas, SQL en BigQuery
- **Destino:** BigQuery, Cloud Storage

ETL vs ELT

Enfoque	Proceso	Cuándo usarlo
ETL	Extract → Transform → Load	Datos sucios, necesitas limpiar antes de cargar
ELT	Extract → Load → Transform	Warehouses modernos (BigQuery). Cargas todo y transformas con SQL

Con BigQuery usaremos **ELT**: cargamos datos brutos y usamos SQL para transformarlos dentro del warehouse.

Pipeline 1: Carga diaria desde CSV a BigQuery

```
"""
pipeline_carga_csv.py
Carga archivos CSV desde una carpeta local a BigQuery.
"""

from google.cloud import bigquery
import pandas as pd
import glob
from datetime import datetime

PROJECT = "techstore-analytics"
DATASET = "techstore_raw"
```

```

CARPETA_CSV = "data/ventas_diarias/"

cliente = bigquery.Client(project=PROJECT)

# Asegurar que el dataset existe
dataset_ref = cliente.dataset(DATASET)
try:
    cliente.get_dataset(dataset_ref)
except Exception:
    dataset = bigquery.Dataset(dataset_ref)
    dataset.location = "us-central1"
    cliente.create_dataset(dataset)
    print(f"Dataset {DATASET} creado")

# Procesar cada CSV
archivos = glob.glob(f"{CARPETA_CSV}*.csv")
print(f"Archivos encontrados: {len(archivos)}")

for archivo in archivos:
    nombre_tabla = f"ventas_{datetime.now().strftime('%Y%m%d')}"

    df = pd.read_csv(archivo)
    print(f"Cargando {archivo}: {len(df)} filas")

    tabla_id = f"{DATASET}.{nombre_tabla}"
    job_config = bigquery.LoadJobConfig(
        write_disposition="WRITE_APPEND",
        autodetect=True,
    )

    job = cliente.load_table_from_dataframe(df, tabla_id, job_config=job_config)
    job.result()
    print(f" Cargadas {job.output_rows} filas en {tabla_id}")

print("Pipeline completado.")

```

Pipeline 2: Transformación con SQL en BigQuery

Una vez que los datos brutos están en BigQuery, creamos tablas transformadas:

```

"""
pipeline_transformaciones.py
Ejecuta transformaciones SQL en BigQuery.
"""

from google.cloud import bigquery

PROJECT = "techstore-analytics"
DATASET_RAW = "techstore_raw"
DATASET_CURATED = "techstore"

cliente = bigquery.Client(project=PROJECT)

```

```

TRANSFORMACIONES = [
    # Ventas diarias consolidadas
    f"""
    CREATE OR REPLACE TABLE {DATASET_CURATED}.ventas_diarias AS
    SELECT
        order_date,
        COUNT(*) AS pedidos,
        COUNT(DISTINCT customer_id) AS clientes_unicos,
        ROUND(SUM(total), 2) AS ingresos,
        ROUND(AVG(total), 2) AS ticket_promedio
    FROM {DATASET_RAW}.orders
    GROUP BY order_date
    ORDER BY order_date
    """,

    # Resumen por categoría
    f"""
    CREATE OR REPLACE TABLE {DATASET_CURATED}.resumen_categorias AS
    SELECT
        p.category,
        COUNT(DISTINCT oi.order_id) AS pedidos,
        SUM(oi.quantity) AS unidades_vendidas,
        ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos,
        ROUND(AVG(oi.unit_price), 2) AS precio_promedio
    FROM {DATASET_RAW}.order_items AS oi
    JOIN {DATASET_RAW}.products AS p ON oi.product_id = p.product_id
    GROUP BY p.category
    ORDER BY ingresos DESC
    """,
]

for i, query in enumerate(TRANSFORMACIONES, 1):
    print(f"Ejecutando transformación {i}...")
    job = cliente.query(query)
    job.result()
    print(f"  Completada ({job.total_bytes_processed / 1e6:.2f} MB procesados)")

print("Todas las transformaciones completadas.")

```

Pipeline 3: Incremental con checkpoint

Para procesar solo datos nuevos, necesitas un checkpoint (última fecha procesada):

```

"""
pipeline_incremental.py
Carga incremental: solo procesa datos nuevos desde la última ejecución.
"""

from google.cloud import bigquery
import pandas as pd
from datetime import datetime, timedelta

```

```

import json
import os

PROJECT = "techstore-analytics"
DATASET = "techstore"
CHECKPOINT_FILE = "checkpoint.json"

cliente = bigquery.Client(project=PROJECT)

def leer_checkpoint():
    if os.path.exists(CHECKPOINT_FILE):
        with open(CHECKPOINT_FILE) as f:
            data = json.load(f)
            return datetime.fromisoformat(data["ultima_ejecucion"])
    return datetime.now() - timedelta(days=7)

def guardar_checkpoint(fecha):
    with open(CHECKPOINT_FILE, "w") as f:
        json.dump({"ultima_ejecucion": fecha.isoformat()}, f)

# Leer desde dónde continuar
ultima_ejecucion = leer_checkpoint()
fecha_inicio = ultima_ejecucion.strftime("%Y-%m-%d")
fecha_fin = datetime.now().strftime("%Y-%m-%d")

print(f"Procesando pedidos desde {fecha_inicio} hasta {fecha_fin}")

# Consultar solo datos nuevos
query = f"""
    SELECT *
    FROM {DATASET}.orders
    WHERE order_date > '{fecha_inicio}'
        AND order_date <= '{fecha_fin}'
    """

df = cliente.query(query).to_dataframe()
print(f"Nuevos pedidos encontrados: {len(df)}")

# Aquí iría la lógica de transformación/carga
# ...

# Guardar checkpoint
guardar_checkpoint(datetime.now())
print("Pipeline incremental completado.")

```

Orchestración con Cloud Composer (Airflow)

Para pipelines en producción, necesitas un orquestador. Google Cloud ofrece **Cloud Composer**

(Apache Airflow gestionado):

```
"""
DAG de ejemplo para Cloud Composer.
Este archivo se sube al bucket de Composer.
"""

from airflow import DAG
from airflow.operators.python import PythonOperator
from google.cloud import bigquery
from datetime import datetime, timedelta

default_args = {
    "owner": "techstore",
    "depends_on_past": False,
    "start_date": datetime(2024, 1, 1),
    "email_on_failure": True,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

def ejecutar_transformaciones():
    cliente = bigquery.Client()
    queries = [
        "CREATE OR REPLACE TABLE techstore.ventas_diarias AS ...",
        "CREATE OR REPLACE TABLE techstore.resumen_categorias AS ...",
    ]
    for query in queries:
        cliente.query(query).result()

with DAG(
    "techstore_pipeline_diario",
    default_args=default_args,
    description="Pipeline diario de TechStore",
    schedule_interval="0 6 * * *", # Todos los días a las 6 AM
    catchup=False,
) as dag:

    t1 = PythonOperator(
        task_id="ejecutar_transformaciones",
        python_callable=ejecutar_transformaciones,
    )

    t1
```

Buenas prácticas para pipelines

1. **Idempotencia:** ejecutar el pipeline N veces debe dar el mismo resultado
2. **Monitoreo:** registra cuántas filas se procesaron y cuánto tomó

3. **Alertas:** notificaciones si el pipeline falla (email, Slack)
4. **Versionado:** las transformaciones SQL deben estar en control de versiones
5. **Tests:** prueba cada etapa del pipeline con datos pequeños
6. **Documentación:** cada tabla transformada debe tener definición y linaje

Ejercicios

1. ¿Cuál es la diferencia entre ETL y ELT? ¿Cuál prefieres para BigQuery?
2. Crea un script Python que lea un CSV y lo cargue a BigQuery
3. ¿Qué es un checkpoint y por qué es importante en pipelines incrementales?
4. Diseña un pipeline ELT para TechStore que genere una tabla de ventas por vendedor
5. ¿Qué es la idempotencia? ¿Cómo la lograrías en una tabla BigQuery?
6. Escribe una transformación SQL que calcule el top 3 productos por mes
7. ¿Para qué sirve Cloud Composer? ¿Qué alternativa más simple existe?
8. Crea un script que verifique que dos tablas en BigQuery tienen el mismo número de filas
9. ¿Qué metadatos registrarías de cada ejecución de pipeline?
10. Implementa un pipeline que cargue datos solo del último mes usando checkpoint

Capítulo 13: Looker Studio — Conectando Datos Cloud

¿Qué es Looker Studio?

Looker Studio (antes Google Data Studio) es una herramienta gratuita de **business intelligence** y **visualización de datos** de Google. Te permite crear dashboards interactivos conectados a diversas fuentes de datos, incluyendo BigQuery.

Características principales:

- **Gratuito:** sin coste para el usuario (solo pagas las consultas a BigQuery)
- **Colaborativo:** edición en tiempo real como Google Docs
- **Conectores nativos:** BigQuery, Google Sheets, Cloud Storage, y 1000+ conectores
- **Interactivo:** filtros cruzados, drill-down, parámetros
- **Programático:** puedes automatizar informes con la API

Looker Studio vs otras herramientas

Herramienta	Precio	Curva aprendizaje	Ideal para
Looker Studio	Gratis	Baja	Dashboards rápidos, equipos pequeños/medianos
Tableau	\$\$\$	Media	BI corporativo, visualizaciones complejas
Power BI	\$\$	Media	Ecosistema Microsoft
Metabase	Gratis (self-hosted)	Baja	Dashboards internos, equipos técnicos
Grafana	Gratis	Media	Series de tiempo, monitoreo

Looker Studio es la opción ideal para empezar: cero inversión, integración directa con BigQuery, y resultados profesionales en horas.

Conectar BigQuery a Looker Studio

Paso 1: Crear un informe nuevo

1. Ve a <https://lookerstudio.google.com>
2. Haz clic en "Informe en blanco"
3. Se abre el panel de "Añadir datos al informe"

Paso 2: Seleccionar conector BigQuery

1. En la lista de conectores, busca y selecciona **BigQuery**
2. Autoriza el acceso si es la primera vez
3. Configura la conexión:
 - **Proyecto:** `techstore-analytics` (o tu Project ID)
 - **Dataset:** `techstore`
 - **Tabla:** `orders`
4. Haz clic en "Añadir"

Paso 3: Primer gráfico

1. En el panel derecho, selecciona "Gráfico de series temporales"
2. Configura:
 - **Dimensión:** `order\_date` (eje X)
 - **Métrica:** `total` SUM (eje Y)
3. En la barra de herramientas, ajusta el periodo a "Últimos 12 meses"
4. ¡Ya tienes tu primer gráfico conectado a BigQuery!

Consultas personalizadas en Looker Studio

En lugar de conectar una tabla completa, puedes escribir SQL personalizado:

1. Al añadir datos, selecciona **BigQuery** "Consulta personalizada"
2. Escribe tu SQL:

```
SELECT
  DATE_TRUNC(order_date, MONTH) AS mes,
  COUNT(*) AS pedidos,
  ROUND(SUM(total), 2) AS ingresos,
  ROUND(AVG(total), 2) AS ticket_promedio
FROM techstore.orders
GROUP BY mes
ORDER BY mes;
```

3. Looker Studio tratará el resultado como una tabla virtual

Componentes básicos de Looker Studio

Dimensiones y métricas

- **Dimensión:** campo descriptivo/categorico (fecha, nombre, ciudad)
- **Métrica:** campo numérico que se agrega (SUM, COUNT, AVG, MIN, MAX)

Tipos de gráficos

Gráfico	Cuándo usarlo
Serie temporal	Evolución en el tiempo
Barra	Comparar categorías
Torta/Donut	Proporciones (pocas categorías)
Tabla	Datos detallados

Tabla pivotante	Resumen por 2 dimensiones
Mapa	Datos geográficos
Tarjeta de puntuación	Un número clave (KPI)
Medidor	Progreso vs objetivo
Diagrama de dispersión	Correlación entre 2 métricas
Gráfico de área apilada	Composición en el tiempo

Diseñando tu primer dashboard

Vamos a crear un dashboard de ventas para TechStore:

Panel 1: KPIs principales

Añade 4 tarjetas de puntuación:

Tarjeta	Métrica	Agregación
Ingresos totales	total	SUM
Pedidos totales	order_id	COUNT
Ticket promedio	total	AVG
Clientes únicos	customer_id	COUNT DISTINCT

Panel 2: Evolución mensual

Gráfico de series temporales:

- **Dimensión:** order_date (día)
- **Métrica:** total SUM
- **Desglose:** category (línea por categoría)

Panel 3: Top productos

Gráfico de barras:

- **Dimensión:** product_name (requiere JOIN con products)
- **Métrica:** quantity SUM
- **Orden:** descendente
- **Límite:** 10

Panel 4: Mapa de clientes

Gráfico de mapa:

- **Dimensión:** city (requiere JOIN con customers)
- **Métrica:** total SUM
- **Estilo:** burbujas proporcionales

Filtros en Looker Studio

Los filtros permiten segmentar los datos sin modificar las consultas:

Filtro a nivel de informe

1. En el menú, "Añadir un control" "Selector de periodo"

2. Configura el rango de fechas
3. Todos los gráficos conectados se actualizan automáticamente

Filtro a nivel de gráfico

1. Selecciona un gráfico
2. En propiedades "Filtro" "Añadir filtro"
3. Ejemplo: ``category = 'Portátiles'``

Filtro por control desplegable

1. "Añadir un control" "Lista desplegable"
2. Dimensión: ``category``
3. Los usuarios pueden seleccionar categorías para filtrar

Compartir dashboards

1. Haz clic en "Compartir" (arriba a la derecha)
2. Opciones:
 - **Personas específicas:** invita por email (requiere edición o solo lectura)
 - **Cualquiera con el enlace:** público o privado
3. También puedes **incrustar** el informe en una web (opción "Incrustar")

Ejercicios

1. Crea un informe en Looker Studio y conéctalo a BigQuery (tabla orders)
2. Añade un gráfico de series temporales con ventas por día
3. Crea un gráfico de barras con las 5 categorías más vendidas
4. ¿Qué diferencia hay entre una dimensión y una métrica?
5. Añade un filtro de fecha a tu dashboard y verifica que los gráficos se actualicen
6. Crea una tarjeta de puntuación que muestre el total de clientes únicos
7. Conecta una consulta personalizada (SQL) en lugar de una tabla completa
8. Añade un mapa con las ventas por ciudad
9. Comparte tu informe como "solo lectura" con un compañero
10. ¿Qué ventajas tiene Looker Studio frente a crear gráficos con matplotlib?

Capítulo 14: Looker Studio Avanzado — Parámetros, Cálculos y Filtros

Campos calculados

Los **campos calculados** te permiten crear nuevas métricas o dimensiones usando fórmulas, sin modificar los datos en BigQuery.

Crear un campo calculado

1. En la fuente de datos, haz clic en "Añadir un campo"
2. Escribe el nombre y la fórmula
3. El campo estará disponible en todos los gráficos

Fórmulas básicas

```
// Margen de beneficio (asumiendo 40% de coste)
(SUM(total) * 0.4) / SUM(total) * 100

// Ticket promedio manual
SUM(total) / COUNT(order_id)

// Descuento aplicado (si precio_original > total)
(SUM(precio_original) - SUM(total)) / SUM(precio_original) * 100

// Edad del cliente a partir de fecha
DATE_DIFF(CURRENT_DATE(), fecha_nacimiento, YEAR)

// Día de la semana
FORMAT_DATETIME('%A', order_date)

// Categorización manual
CASE
  WHEN SUM(total) >= 1000 THEN 'Alto'
  WHEN SUM(total) >= 500 THEN 'Medio'
  ELSE 'Bajo'
END
```

Fórmulas de agregación

```
// Promedio móvil (7 días)
SUM(SUM(total)) OVER (ORDER BY order_date ROWS 6 PRECEDING) / 7
```

```
// Diferencia vs mes anterior
SUM(total) - SUM(SUM(total)) OVER (ORDER BY order_date ROWS 1 PRECEDING)

// Porcentaje del total
SUM(total) / SUM(SUM(total)) OVER () * 100
```

Parámetros

Los **parámetros** permiten que el usuario controle aspectos del dashboard sin modificar filtros manualmente.

Usar parámetros para objetivos

1. "Añadir un control" "Parámetro de entrada"
2. Nombre: `objetivo\_ingresos`
3. Tipo: Número
4. Valor por defecto: `100000`

Luego crea un campo calculado que use el parámetro:

```
// Porcentaje de cumplimiento
SUM(total) / objetivo_ingresos * 100
```

Parámetro para top N

1. Crea un parámetro: `top\_n` Número Valor: 10
2. En un campo calculado:

```
// Ranking por ingresos
RANK() OVER (ORDER BY SUM(total) DESC) <= top_n
```

3. Usa este campo como filtro booleano en el gráfico

Blending de datos

El **blending** combina datos de múltiples fuentes sin JOINS en SQL, similar a un VLOOKUP de Excel:

1. "Añadir datos" "Combinar datos" (Blend)
2. Selecciona la fuente izquierda (ej: orders)
3. "Añadir otra fuente" (ej: customers)
4. Define la clave de unión: `customer\_id`
5. Selecciona las métricas y dimensiones de cada fuente

Blending vs JOIN en BigQuery

Blending	JOIN SQL
Se configura en UI	Se escribe en SQL
Menos flexible	Totalmente flexible
Bueno para uniones simples	Necesario para transformaciones complejas
Limitado a 5 fuentes	Sin límite práctico

Filtros avanzados

Filtros con expresiones regulares

```
REGEXP_MATCH(customer.email, '.*@gmail\\.com$')
```

Filtros por rango dinámico

```
// Productos con precio sobre el promedio  
unit_price > AVG(unit_price)
```

Filtros entre dashboards

Puedes pasar filtros entre páginas de un mismo informe:

1. Añade un control desplegable en la página 1
2. En la página 2, el filtro se aplica automáticamente (misma fuente de datos)

Comunidad de conectores y visualizaciones

Looker Studio tiene una **galería de socios** con visualizaciones adicionales:

- **Gantt Chart:** cronogramas
- **Word Cloud:** nubes de palabras
- **Heatmap Calendar:** calendario de calor
- **Sankey Diagram:** flujos
- **Bullet Chart:** barras con objetivo

Para instalarlos:

1. En el informe, "Editar" "Administrar visualizaciones adicionales"
2. Explora la galería
3. Añade las que necesites

Optimización de dashboards con BigQuery

Cache

Looker Studio cachea resultados de BigQuery por 12 horas por defecto. Puedes ajustar esto en la configuración de la fuente de datos.

Consultas eficientes

- Usa agregaciones precalculadas (tablas materializadas)
- Limita el rango de fechas por defecto
- Evita conexiones directas a tablas sin resumir
- Programa actualizaciones en horas de menor coste

Programación de informes

1. En la fuente de datos, "Programación de actualización"
2. Frecuencia: cada 4 horas, diario, semanal
3. Configura la hora para evitar picos de coste

Ejemplo: Dashboard de rendimiento de ventas

Crea un dashboard completo con:

Página 1: Resumen ejecutivo

- 4 tarjetas KPI (ingresos, pedidos, ticket promedio, clientes activos)
- Serie temporal de ingresos vs mes anterior
- Tabla con top 10 productos
- Mapa de ventas por ciudad

Página 2: Análisis detallado

- Tabla pivotante (categoría × mes)
- Filtro por vendedor (empleado)
- Gráfico de barras apiladas (categoría por mes)
- Parámetro para ajustar el objetivo de ingresos

Página 3: Cohortes y retención

- Heatmap de cohortes mensuales
- Curva de retención (clientes que repiten)
- Filtro de fecha para seleccionar período

Ejercicios

1. Crea un campo calculado que categorice pedidos como "Pequeño", "Mediano", "Grande" según el total
2. Añade un parámetro numérico para ajustar el objetivo de ingresos
3. Usa un filtro REGEXP_MATCH para mostrar solo clientes con email corporativo
4. Crea un blend entre orders y employees para mostrar ventas por vendedor
5. ¿Qué diferencia hay entre un filtro y un parámetro en Looker Studio?
6. Añade una visualización de terceros (ej: Word Cloud o Sankey)
7. Programa la actualización diaria de tu fuente de datos BigQuery
8. Crea un campo calculado con CASE para clasificar ciudades por región
9. Usa un parámetro para que el usuario seleccione el TOP N de productos
10. Crea un dashboard de 2 páginas con filtro cruzado entre ellas

Checkpoint 3: Dashboard Cloud de Ventas TechStore

Objetivo

Has aprendido SQL analítico en BigQuery, data pipelines con Python y Looker Studio. Ahora es momento de integrar todo: **crear un dashboard profesional en Looker Studio** conectado a BigQuery, con múltiples páginas, filtros interactivos y campos calculados.

Requisitos

- Dataset `techstore` migrado a BigQuery (Checkpoint 2 completado)
- Cuenta de Looker Studio (lookerstudio.google.com)
- Conexión a BigQuery configurada

Paso 1: Preparar las consultas base

Crea una **consulta personalizada** en Looker Studio que sirva como fuente principal del dashboard. Esta consulta debe devolver datos limpios y enriquecidos:

```
-- Vista principal del dashboard
SELECT
  o.order_id,
  o.order_date,
  o.status,
  o.total,
  EXTRACT(YEAR FROM o.order_date) AS anio,
  EXTRACT(MONTH FROM o.order_date) AS mes,
  FORMAT_DATE('%Y-%m', o.order_date) AS anio_mes,
  EXTRACT(QUARTER FROM o.order_date) AS trimestre,
  EXTRACT(DAYOFWEEK FROM o.order_date) AS dia_semana,
  FORMAT_DATE('%A', o.order_date) AS nombre_dia_semana,
  c.customer_id,
  c.name AS customer_name,
  c.city,
  c.country,
  c.email,
  e.employee_id,
  e.name AS employee_name,
```

```

    e.position AS employee_position
FROM techstore.orders AS o
JOIN techstore.customers AS c ON o.customer_id = c.customer_id
JOIN techstore.employees AS e ON o.employee_id = e.employee_id
WHERE o.order_date >= '2023-01-01'

```

Además, crea una segunda consulta para el detalle de productos:

```

-- Vista de productos por pedido
SELECT
    oi.order_id,
    oi.product_id,
    p.name AS product_name,
    p.category,
    p.supplier,
    oi.quantity,
    oi.unit_price,
    (oi.quantity * oi.unit_price) AS line_total,
    o.order_date,
    o.customer_id
FROM techstore.order_items AS oi
JOIN techstore.products AS p ON oi.product_id = p.product_id
JOIN techstore.orders AS o ON oi.order_id = o.order_id
WHERE o.order_date >= '2023-01-01'

```

Paso 2: Diseñar la estructura del dashboard

Crea un informe con **4 páginas**:

Página 1: Resumen Ejecutivo

- **Título:** "TechStore — Panel de Ventas"
- **KPIs** (tarjetas de puntuación): Ingresos totales, Pedidos, Ticket promedio, Clientes únicos
- **Gráfico:** Serie temporal de ingresos mensuales
- **Gráfico:** Top 10 productos más vendidos (barras horizontales)
- **Filtro:** Selector de fecha (a nivel de informe)

Página 2: Análisis Geográfico

- **Gráfico:** Mapa de ventas por ciudad
- **Gráfico:** Barras con top 10 ciudades
- **Tabla:** Detalle de ventas por país/ciudad
- **Filtro:** Desplegable de país

Página 3: Rendimiento por Vendedor

- **Gráfico:** Barras apiladas de ingresos por vendedor
- **Gráfico:** Evolución mensual por vendedor (líneas múltiples)
- **Tabla:** Rankings de vendedores

- **Filtro:** Desplegable de vendedor

Página 4: Detalle de Productos

- **Gráfico:** Torta de ingresos por categoría
- **Tabla pivotante:** Categoría × Mes (ingresos)
- **Gráfico:** Dispersión (precio unitario vs cantidad vendida)
- **Campo calculado:** Margen estimado (precio * 0.4) como columna adicional

Paso 3: Implementar campos calculados

Crea estos campos calculados en tu fuente de datos:

```
// Ticket promedio (manual)
Ingreso por pedido
SUM(total)

// Día de la semana (texto)
FORMAT_DATETIME('%A', order_date)

// Mes (texto corto)
FORMAT_DATETIME('%b %Y', order_date)

// Categoría de ticket
CASE
  WHEN total >= 1000 THEN 'Alto'
  WHEN total >= 500 THEN 'Medio'
  ELSE 'Bajo'
END

// Trimestre fiscal
CONCAT('Q', CAST(EXTRACT(QUARTER FROM order_date) AS TEXT), ' - ', CAST(EXTRACT(YEAR FROM order_date) AS TEXT))
```

Paso 4: Añadir interactividad

1. **Selector de fecha** en todas las páginas (control de rango)
2. **Desplegable de categoría** para filtrar productos
3. **Desplegable de vendedor** en página 3
4. **Filtro cruzado:** al hacer clic en un gráfico, se filtran los demás
5. **Parámetro "Objetivo mensual"** con valor por defecto \$50,000
 - Campo calculado: ``SUM(total) / objetivo_mensual * 100``

Paso 5: Personalizar apariencia

1. Tema de colores corporativo (azul oscuro + naranja)
2. Logo de TechStore (opcional — puedes usar un icono)
3. Títulos descriptivos en cada gráfico
4. Formato de moneda (\$) en todas las métricas de ingresos

5. Fondo blanco/límpio, sin elementos distractores

Paso 6: Compartir y programar

1. Comparte el informe con "Cualquiera con el enlace puede ver"
2. Programa la actualización de la fuente de datos BigQuery (diaria a las 6 AM)
3. Descarga el informe como PDF

Entregables del Checkpoint 3

Al completar este checkpoint deberías tener:

- ☐ Informe en Looker Studio con 4 páginas
- ☐ Conexión a BigQuery con 2 consultas personalizadas
- ☐ KPIs principales en la página de resumen
- ☐ Mapa geográfico funcionando
- ☐ Análisis por vendedor
- ☐ Detalle de productos con tabla pivotante
- ☐ Mínimo 3 campos calculados
- ☐ Filtros interactivos (fecha, categoría, vendedor)
- ☐ Parámetro de objetivo mensual
- ☐ Informe compartido (enlace de solo lectura)

Preguntas de reflexión

1. ¿Cuánto tiempo te tomó crear el dashboard? ¿Qué fue lo más complejo?
2. ¿Qué información nueva descubriste sobre TechStore al ver los datos visualmente?
3. ¿Qué ventaja tiene un dashboard interactivo frente a un informe PDF estático?
4. Si tuvieras que presentar este dashboard a la dirección, ¿qué página mostrarías primero?
5. ¿Cómo mejorarías este dashboard con datos adicionales (ej: inventario, costes, devoluciones)?

¡Felicidades! Has creado tu primer dashboard cloud conectado a BigQuery. En el próximo proyecto automatizarás todo el pipeline para que se actualice sin intervención manual.

Capítulo 15: Automatización Serverless con Cloud Functions

¿Qué es serverless?

Serverless no significa "sin servidores", sino "sin que tú gestionas servidores". Escribes código, lo subes y el proveedor (Google Cloud) se encarga de ejecutarlo, escalarlo y mantenerlo.

Ventajas:

- **Sin aprovisionamiento:** no eliges máquinas ni capacidades
- **Escalado automático:** de 0 a miles de ejecuciones simultáneas
- **Pago por uso:** solo pagas mientras tu código se ejecuta
- **Alta disponibilidad:** Google replica tu función en múltiples zonas

Cloud Functions

Cloud Functions es el servicio serverless de Google Cloud. Ejecutas código Python (o Node.js, Go, Java, .NET, Ruby) en respuesta a eventos.

Tipos de Cloud Functions

Tipo	Disparador	Uso típico
HTTP	Petición HTTP	APIs, webhooks, endpoints
Cloud Storage	Evento en GCS	Procesar archivos al subirse
Pub/Sub	Mensaje en cola	Pipelines asíncronos, eventos
Firestore	Cambio en documento	Reaccionar a cambios en DB
Cloud Scheduler	Cron programado	Tareas periódicas

Tu primera función HTTP

1. Crear la función

```
"""
funcion_http/main.py
Cloud Function HTTP: endpoint de saludo.
"""

def saludar(request):
```

```

"""Responde a peticiones HTTP."""
nombre = request.args.get("nombre", "Mundo")
return f"Hola, {nombre}! Bienvenido a TechStore Analytics."

```

2. requirements.txt

```

functions-framework==3.*
google-cloud-bigquery
pandas

```

3. Desplegar

```

# Desplegar con gcloud
gcloud functions deploy saludar \
    --runtime python311 \
    --trigger-http \
    --allow-unauthenticated \
    --region us-central1

# Probar
curl https://REGION-PROJECT_ID.cloudfunctions.net/saludar?nombre=Alex

```

Función: Consultar ventas del día

```

"""
funcion_ventas_diarias/main.py
Cloud Function: devuelve las ventas del día actual.
"""

from google.cloud import bigquery
from datetime import date

PROJECT = "techstore-analytics"
DATASET = "techstore"

cliente = bigquery.Client(project=PROJECT)

def ventas_hoy(request):
    """Devuelve resumen de ventas del día."""
    hoy = date.today().isoformat()

    query = f"""
        SELECT
            COUNT(*) AS pedidos,
            ROUND(SUM(total), 2) AS ingresos,
            COUNT(DISTINCT customer_id) AS clientes
        FROM {DATASET}.orders
        WHERE DATE(order_date) = '{hoy}'
    """

```

```

df = cliente.query(query).to_dataframe()

return {
    "fecha": hoy,
    "pedidos": int(df["pedidos"].iloc[0]),
    "ingresos": float(df["ingresos"].iloc[0]),
    "clientes": int(df["clientes"].iloc[0]),
}

```

Función disparada por Cloud Storage

```

"""
funcion_procesar_csv/main.py
Cloud Function: cuando se sube un CSV a GCS, lo procesa y carga a BigQuery.
"""

from google.cloud import bigquery, storage
import pandas as pd
import os

PROJECT = "techstore-analytics"
DATASET = "techstore_raw"

cliente_bq = bigquery.Client(project=PROJECT)
cliente_storage = storage.Client()

def procesar_csv(event, context):
    """Disparada por evento en Cloud Storage."""
    archivo = event["name"]
    bucket = event["bucket"]

    print(f"Procesando: gs://{bucket}/{archivo}")

    # Descargar archivo a directorio temporal
    ruta_local = f"/tmp/{os.path.basename(archivo)}"
    blob = cliente_storage.bucket(bucket).blob(archivo)
    blob.download_to_filename(ruta_local)

    # Leer CSV
    df = pd.read_csv(ruta_local)
    print(f"Filas: {len(df)}")

    # Cargar a BigQuery
    nombre_tabla = os.path.splitext(os.path.basename(archivo))[0]
    tabla_id = f"{DATASET}.{nombre_tabla}"

    job_config = bigquery.LoadJobConfig(
        write_disposition="WRITE_APPEND",
        autodetect=True,
    )

```

```

job = cliente_bq.load_table_from_dataframe(df, tabla_id, job_config=job_config)
job.result()

print(f"Cargadas {job.output_rows} filas en {tabla_id}")

# Opcional: mover archivo procesado
blob_copy = cliente_storage.bucket(bucket).blob(f"procesados/{archivo}")
blob_copy.rewrite(blob)
blob.delete()

return f"OK: {len(df)} filas procesadas"

```

Función con Pub/Sub

```

"""
funcion_pubsub/main.py
Cloud Function disparada por mensaje Pub/Sub.
"""

import base64
import json
from google.cloud import bigquery

PROJECT = "techstore-analytics"
DATASET = "techstore"
cliente = bigquery.Client(project=PROJECT)

def procesar_evento(event, context):
    """Procesa un mensaje Pub/Sub con datos de venta."""

    # Decodificar mensaje
    datos = base64.b64decode(event["data"]).decode("utf-8")
    venta = json.loads(datos)

    print(f"Procesando venta: {venta}")

    # Insertar en BigQuery
    query = f"""
        INSERT INTO {DATASET}.ventas_streaming
        (order_id, customer_id, total, timestamp)
        VALUES
        (@order_id, @customer_id, @total, CURRENT_TIMESTAMP())
    """

    job_config = bigquery.QueryJobConfig(
        query_parameters=[
            bigquery.ScalarQueryParameter("order_id", "INT64", venta["order_id"]),
            bigquery.ScalarQueryParameter("customer_id", "INT64", venta["customer_id"]),
            bigquery.ScalarQueryParameter("total", "FLOAT64", venta["total"]),

```

```

    ]
)

cliente.query(query, job_config=job_config).result()
print("Venta registrada en BigQuery.")

```

Despliegue y monitorización

Comandos gcloud útiles

```

# Listar funciones
gcloud functions list

# Ver logs
gcloud functions logs read saludar

# Describir función
gcloud functions describe saludar

# Eliminar función
gcloud functions delete saludar

```

Logs y errores

Cloud Functions se integra con **Cloud Logging** para registrar automáticamente:

- Salida de ``print()``
- Excepciones no capturadas
- Métricas de ejecución (duración, memoria, invocaciones)

Buenas prácticas

1. **Función atómica:** cada función hace UNA cosa bien
2. **Timeout:** por defecto 60s, máximo 540s (9 min). Funciones largas Cloud Run
3. **Idempotencia:** procesar el mismo mensaje dos veces no debe causar duplicados
4. **Mínimas dependencias:** menos dependencias = menos frialdad (cold start)
5. **Variables de entorno:** para configuraciones sensibles (no hardcodear)
6. **Retry:** Pub/Sub reintenta automáticamente; HTTP no

Cold starts

La primera invocación después de un periodo de inactividad puede tardar más (la plataforma inicia tu entorno). Para evitarlo:

- Mantén una o dos instancias en espera (``min_instances``)
- Usa funciones más ligeras
- Para cargas críticas, considera Cloud Run

Ejercicios

1. Crea y despliega una Cloud Function HTTP que devuelva "Hola, TechStore"
2. ¿Qué es un cold start? ¿Cómo mitigarlo?
3. Crea una función que consulte el total de pedidos de hoy en BigQuery y los devuelva como JSON
4. ¿Cuál es la diferencia entre una función HTTP y una función disparada por Storage?
5. Despliega una función que procese un CSV cuando se suba a un bucket de GCS
6. ¿Para qué sirve Pub/Sub? Nombra 2 casos de uso en data analytics
7. Crea una función que reciba un mensaje Pub/Sub y registre la venta en BigQuery
8. ¿Cómo monitoreas errores en Cloud Functions?
9. Configura variables de entorno para las credenciales de BigQuery
10. ¿Cuándo usarías Cloud Run en lugar de Cloud Functions? (investiga)

Capítulo 16: Schedule Queries y Pipelines Automáticos

Automatización sin servidores

No todo necesita Cloud Functions. BigQuery ofrece herramientas de automatización integradas que no requieren código:

1. **Schedule Queries:** ejecuta consultas SQL automáticamente en un horario
2. **Cloud Scheduler:** programa cualquier tarea (incluyendo invocar Cloud Functions)
3. **Workflows:** orquesta servicios de GCP en secuencia

Scheduled Queries en BigQuery

Las **Scheduled Queries** ejecutan automáticamente consultas SQL en un horario definido y guardan los resultados en una tabla.

Crear una consulta programada

```
-- Consulta que se ejecutará diariamente
-- Guarda el resumen diario de ventas
CREATE OR REPLACE TABLE techstore.resumen_diario AS
SELECT
  CURRENT_DATE() AS fecha_ejecucion,
  DATE(order_date) AS fecha_venta,
  COUNT(*) AS pedidos,
  COUNT(DISTINCT customer_id) AS clientes_unicos,
  ROUND(SUM(total), 2) AS ingresos,
  ROUND(AVG(total), 2) AS ticket_promedio
FROM techstore.orders
WHERE DATE(order_date) = DATE_SUB(CURRENT_DATE(), INTERVAL 1 DAY)
GROUP BY fecha_venta;
```

Para programarla:

1. En la consola de BigQuery, ejecuta la consulta
2. Haz clic en "Programar" "Nueva programación"
3. Configura:
 - **Nombre:** `resumen\_diario\_techstore`
 - **Frecuencia:** Diario

- **Hora:** 03:00 (mañana, menor coste)
- **Destino:** `techstore.resumen\_diario`
- **Opción de escritura:** `WRITE\_APPEND`

4. Guarda

Consulta recurrente con partición por fecha

```
-- Resumen semanal
CREATE OR REPLACE TABLE techstore.resumen_semanal AS
SELECT
  DATE_TRUNC(order_date, WEEK(MONDAY)) AS semana_inicio,
  DATE_ADD(DATE_TRUNC(order_date, WEEK(MONDAY)), INTERVAL 6 DAY) AS semana_fin,
  COUNT(*) AS pedidos,
  ROUND(SUM(total), 2) AS ingresos,
  ROUND(AVG(total), 2) AS ticket_promedio,
  ROUND(SUM(total) / COUNT(DISTINCT customer_id), 2) AS ingresos_por_cliente
FROM techstore.orders
WHERE order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 12 WEEK)
GROUP BY semana_inicio
ORDER BY semana_inicio;
```

Cloud Scheduler + Cloud Functions

Cloud Scheduler es un servicio de cron gestionado. Puede invocar una Cloud Function HTTP en un horario.

Ejemplo: Informe diario por email

1. Crea una Cloud Function que genere un resumen y lo envíe por email (o lo guarde en GCS)

```
"""
funcion_informe_diario/main.py
Cloud Function: genera informe diario y lo guarda en GCS.
"""

from google.cloud import bigquery, storage
from datetime import date, timedelta
import pandas as pd

PROJECT = "techstore-analytics"
DATASET = "techstore"
BUCKET_INFORMES = "techstore-informes"

cliente_bq = bigquery.Client(project=PROJECT)
cliente_gcs = storage.Client()

def generar_informe_diario(request=None):
    """Genera informe diario y lo guarda en Cloud Storage."""
    ayer = date.today() - timedelta(days=1)
    mes = ayer.strftime("%Y%m")
    dia = ayer.strftime("%Y%m%d")
```

```

# Consultar ventas de ayer
query = f"""
    SELECT
        DATE(order_date) AS fecha,
        COUNT(*) AS pedidos,
        ROUND(SUM(total), 2) AS ingresos,
        p.category,
        COUNT(DISTINCT o.customer_id) AS clientes
    FROM {DATASET}.orders AS o
    JOIN {DATASET}.order_items AS oi ON o.order_id = oi.order_id
    JOIN {DATASET}.products AS p ON oi.product_id = p.product_id
    WHERE DATE(order_date) = '{ayer.isoformat()}'
    GROUP BY fecha, p.category
    ORDER BY ingresos DESC
"""

df = cliente_bq.query(query).to_dataframe()

# Guardar como CSV en GCS
ruta = f"diarios/{mes}/informe_{dia}.csv"
bucket = cliente_gcs.bucket(BUCKET_INFORMES)
blob = bucket.blob(ruta)
blob.upload_from_string(df.to_csv(index=False), content_type="text/csv")

# Guardar también como Excel (usando pandas ExcelWriter)
ruta_excel = f"diarios/{mes}/informe_{dia}.xlsx"
blob_excel = bucket.blob(ruta_excel)
with blob_excel.open("wb") as f:
    df.to_excel(f, index=False, sheet_name="Ventas Diarias")

return f"Informe {dia} generado: {len(df)} categorías"

```

2. Crea un trabajo en Cloud Scheduler:

```

# Crear trabajo que ejecute la función cada día a las 7 AM
gcloud scheduler jobs create http informe-diario \
  --schedule="0 7 * * *" \
  --uri="https://REGION-PROJECT.cloudfunctions.net/generar-informe-diario" \
  --http-method=GET \
  --time-zone="America/Lima" \
  --location=us-central1

```

Workflows: orquestación visual

Workflows permite encadenar servicios de GCP en una secuencia definida como YAML:

```

# workflow_informe_diario.yaml
main:
  steps:
    - step1_generar_datos:
        call: googleapis.bigquery.v2.jobs.query
        args:

```

```

projectId: techstore-analytics
body:
  query: |
    CREATE OR REPLACE TABLE techstore.resumen_diario AS
    SELECT DATE(order_date) AS fecha, COUNT(*) AS pedidos,
           ROUND(SUM(total), 2) AS ingresos
    FROM techstore.orders
    WHERE DATE(order_date) = DATE_SUB(CURRENT_DATE(), INTERVAL 1 DAY)
    GROUP BY fecha

- step2_exportar_csv:
  call: googleapis.bigquery.v2.jobs.query
  args:
    projectId: techstore-analytics
    body:
      query: |
        EXPORT DATA OPTIONS(
          uri='gs://techstore-informes/diario/*.csv',
          format='CSV',
          overwrite=true
        ) AS
        SELECT * FROM techstore.resumen_diario

- step3_enviar_notificacion:
  call: googleapis.pubsub.v1.projects.topics.publish
  args:
    topic: projects/techstore-analytics/topics/informe-generado
    body:
      messages:
        - data: "Informe diario generado exitosamente"

```

```

# Desplegar workflow
gcloud workflows deploy techstore-informe-diario \
  --source=workflow_informe_diario.yaml \
  --location=us-central1

```

Cloud Run: serverless para contenedores

Cloud Run ejecuta contenedores serverless. Es como Cloud Functions pero para aplicaciones completas (no solo una función):

```

# Dockerfile para Cloud Run
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .
CMD ["python", "app.py"]

```

```

"""

```

```

app.py para Cloud Run
API REST con análisis de TechStore.
"""

from flask import Flask, jsonify
from google.cloud import bigquery

app = Flask(__name__)
cliente = bigquery.Client()

@app.route("/")
def home():
    return "TechStore Analytics API – Cloud Run"

@app.route("/kpi")
def kpi():
    query = """
        SELECT
            ROUND(SUM(total), 2) AS ingresos,
            COUNT(*) AS pedidos,
            COUNT(DISTINCT customer_id) AS clientes
        FROM techstore.orders
    """

    df = cliente.query(query).to_dataframe()
    return jsonify(df.to_dict(orient="records")[0])

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)

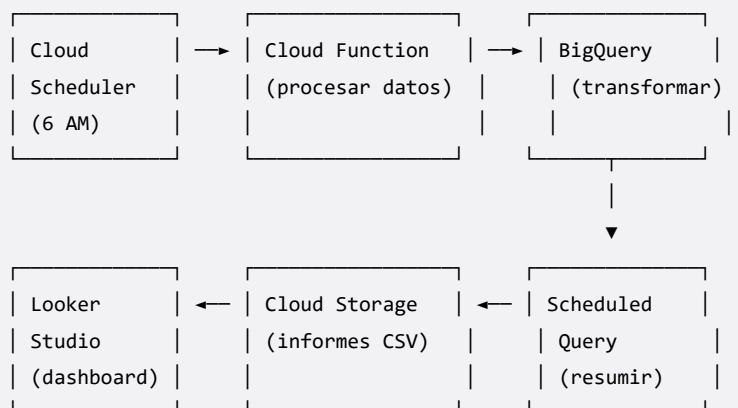
```

```

# Construir y desplegar
gcloud builds submit --tag gcr.io/PROJECT_ID/techstore-api
gcloud run deploy techstore-api \
    --image gcr.io/PROJECT_ID/techstore-api \
    --region us-central1 \
    --allow-unauthenticated

```

Pipeline completamente automatizado



Buenas prácticas de automatización

1. **Empieza simple:** Scheduled Queries antes que Cloud Functions
2. **Logs centralizados:** todo pipeline debe registrar inicio, fin, errores
3. **Alertas:** configura notificaciones en fallos (Pub/Sub Email/Slack)
4. **Costes:** programa tareas pesadas en horas de menor tarifa
5. **Versiones:** usa tags para las funciones (``gcloud functions deploy --entry-point``)
6. **Pruebas:** prueba cada etapa por separado antes de orquestar

Ejercicios

1. Crea una Scheduled Query que genere el resumen diario de ventas
2. ¿Qué diferencia hay entre Cloud Scheduler y Scheduled Queries?
3. Programa un Cloud Scheduler que invoque una función HTTP cada hora
4. ¿Para qué sirve Workflows? Nombra un caso de uso
5. Crea un pipeline que: Schedule Cloud Function BigQuery GCS
6. ¿Cuándo usarías Cloud Run en lugar de Cloud Functions?
7. Configura una alerta por email cuando falle una Scheduled Query
8. Crea una función que genere un informe semanal en Excel y lo guarde en GCS
9. ¿Cómo gestionarías errores en un pipeline de múltiples pasos?
10. Diseña en papel un pipeline completo para TechStore desde la carga hasta el dashboard

Capítulo 17: Proyecto Final — Sistema de Reporting Cloud

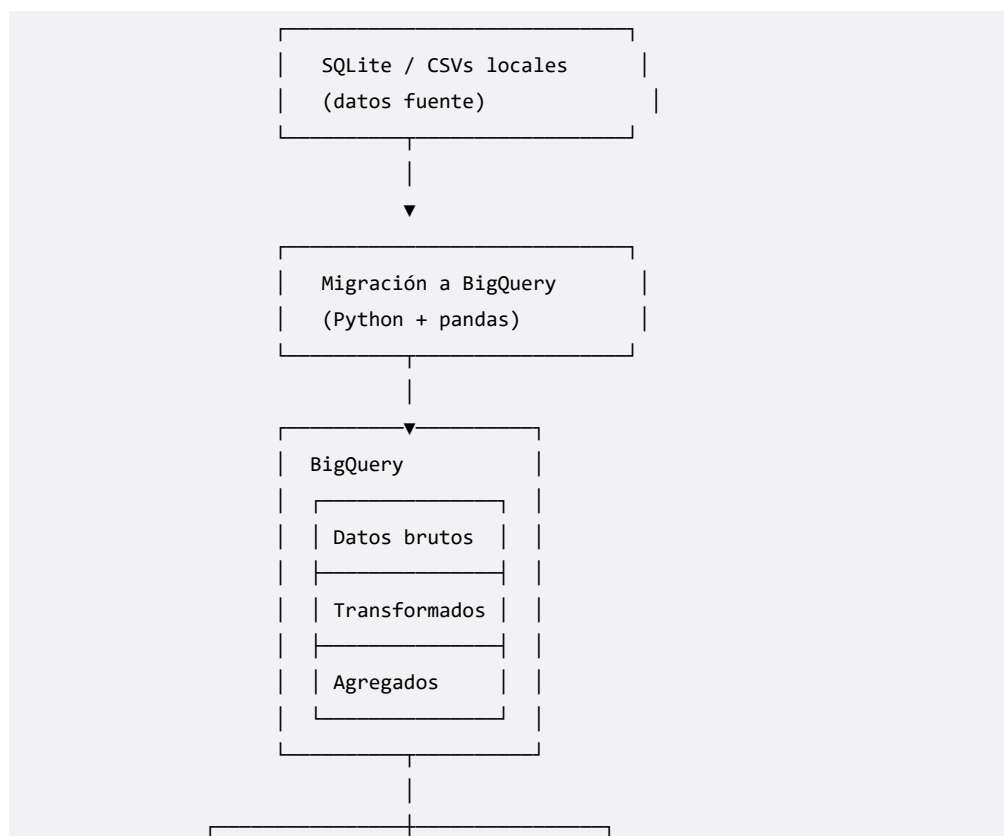
El cuadro completo

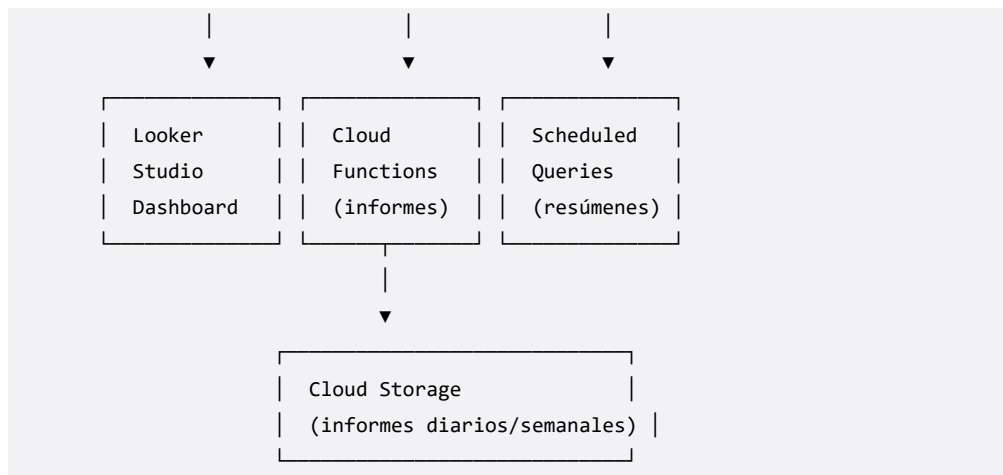
A lo largo de este libro has aprendido:

1. **Python + pandas** para análisis de datos
2. **BigQuery** como data warehouse cloud
3. **Looker Studio** para dashboards interactivos
4. **Cloud Functions y automatización** para pipelines serverless

Ahora es momento de integrarlo todo en un **sistema de reporting cloud** completo y automatizado para TechStore.

Arquitectura del sistema





Componente 1: Base de datos en la nube

Tu dataset `techstore` en BigQuery ya tiene las 5 tablas migradas. Ahora añade tablas derivadas para optimizar el reporting:

```

-- Tabla de métricas diarias (para dashboard rápido)
CREATE OR REPLACE TABLE techstore.metricas_diarias AS
SELECT
  DATE(order_date) AS fecha,
  COUNT(*) AS pedidos,
  COUNT(DISTINCT customer_id) AS clientes_activos,
  ROUND(SUM(total), 2) AS ingresos,
  ROUND(AVG(total), 2) AS ticket_promedio,
  ROUND(SUM(total) / NULLIF(COUNT(DISTINCT customer_id), 0), 2) AS ingresos_por_cliente,
  COUNT(*) / NULLIF(COUNT(DISTINCT customer_id), 0) AS pedidos_por_cliente
FROM techstore.orders
GROUP BY fecha
ORDER BY fecha;

-- Tabla de cohortes (actualizada semanalmente)
CREATE OR REPLACE TABLE techstore.cohortes_semanales AS
WITH primera_compra AS (
  SELECT
    customer_id,
    DATE_TRUNC(MIN(order_date), WEEK(MONDAY)) AS cohorte
  FROM techstore.orders
  GROUP BY customer_id
),
semanas AS (
  SELECT
    pc.customer_id,
    pc.cohorte,
    DATE_TRUNC(o.order_date, WEEK(MONDAY)) AS semana,
    DATE_DIFF(DATE_TRUNC(o.order_date, WEEK(MONDAY)), pc.cohorte, WEEK) AS semana_numero
  FROM techstore.orders AS o
  JOIN primera_compra AS pc ON o.customer_id = pc.customer_id
)
SELECT

```



```

    cohorte,
    semana_numero,
    COUNT(DISTINCT customer_id) AS clientes
FROM semanas
GROUP BY cohorte, semana_numero;

```

Componente 2: Pipeline de actualización

Crea un pipeline automático que se ejecute diariamente:

```

"""
pipeline_completo.py
Pipeline completo de actualización de TechStore.

Ejecuta: transformaciones en BigQuery → exporta informes → notifica.
"""

from google.cloud import bigquery
from datetime import datetime
import logging

PROJECT = "techstore-analytics"
DATASET = "techstore"

# Configurar logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

cliente = bigquery.Client(project=PROJECT)

def ejecutar_query(query, descripcion):
    """Ejecuta una consulta y registra el resultado."""
    logger.info(f"Ejecutando: {descripcion}")
    job = cliente.query(query)
    job.result()
    bytes_procesados = job.total_bytes_processed / 1e6
    logger.info(f"Completado ({bytes_procesados:.2f} MB)")
    return bytes_procesados

def pipeline_diario():
    """Ejecuta todas las transformaciones del pipeline diario."""
    inicio = datetime.now()
    logger.info(f"Iniciando pipeline diario: {inicio.isoformat()}")

    total_bytes = 0

    # Paso 1: Actualizar métricas diarias
    total_bytes += ejecutar_query(f"""
        CREATE OR REPLACE TABLE {DATASET}.metricas_diarias AS
        SELECT

```

```

        DATE(order_date) AS fecha,
        COUNT(*) AS pedidos,
        COUNT(DISTINCT customer_id) AS clientes_activos,
        ROUND(SUM(total), 2) AS ingresos,
        ROUND(AVG(total), 2) AS ticket_promedio
    FROM {DATASET}.orders
    GROUP BY fecha
    ORDER BY fecha
    """ , "Actualizar métricas diarias")

# Paso 2: Actualizar top productos
total_bytes += ejecutar_query(f"""
    CREATE OR REPLACE TABLE {DATASET}.top_productos AS
    SELECT
        p.category,
        p.name AS producto,
        SUM(oi.quantity) AS unidades_vendidas,
        ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos,
        COUNT(DISTINCT oi.order_id) AS pedidos_con_producto
    FROM {DATASET}.order_items AS oi
    JOIN {DATASET}.products AS p ON oi.product_id = p.product_id
    WHERE oi.order_id IN (
        SELECT order_id FROM {DATASET}.orders
        WHERE order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
    )
    GROUP BY p.category, p.name
    ORDER BY ingresos DESC
    """ , "Actualizar top productos")

# Paso 3: Métricas de rendimiento por empleado
total_bytes += ejecutar_query(f"""
    CREATE OR REPLACE TABLE {DATASET}.rendimiento_vendedores AS
    SELECT
        e.employee_id,
        e.name AS vendedor,
        e.position,
        COUNT(o.order_id) AS pedidos,
        ROUND(SUM(o.total), 2) AS ingresos,
        ROUND(AVG(o.total), 2) AS ticket_promedio,
        ROUND(COUNT(o.order_id) / NULLIF(COUNT(DISTINCT DATE(o.order_date)), 0), 1) AS pedidos_por_dia
    FROM {DATASET}.orders AS o
    JOIN {DATASET}.employees AS e ON o.employee_id = e.employee_id
    WHERE o.order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 90 DAY)
    GROUP BY e.employee_id, e.name, e.position
    ORDER BY ingresos DESC
    """ , "Actualizar rendimiento vendedores")

duracion = (datetime.now() - inicio).total_seconds()
logger.info(f"Pipeline completado en {duracion:.1f}s. Total procesado: {total_bytes:.2f} MB")
return {"duracion_seg": duracion, "total_mb": total_bytes}

```

```
if __name__ == "__main__":
    resultado = pipeline_diario()
    print(f"Pipeline ejecutado: {resultado}")
```

Componente 3: Dashboard ejecutivo

Crea un dashboard en Looker Studio con estas páginas:

Página 1: Vista Ejecutiva

- KPIs: Ingresos, Pedidos, Clientes, Ticket promedio (vs mes anterior)
- Serie temporal 12 meses
- Top 10 productos
- Métricas diarias (últimos 30 días)

Página 2: Análisis de Clientes

- Mapa geográfico
- Top 10 ciudades
- Análisis de cohortes (heatmap)
- Clientes nuevos vs recurrentes

Página 3: Operaciones

- Rendimiento de vendedores
- Pedidos por hora/día
- Categorías más vendidas
- Tasa de conversión (pedidos vs clientes)

Componente 4: Notificaciones automáticas

Configura alertas para eventos importantes:

```
"""
alertas_techstore/main.py
Cloud Function: verifica métricas y envía alertas.
"""

from google.cloud import bigquery
import json
import os

PROJECT = "techstore-analytics"
DATASET = "techstore"

cliente = bigquery.Client(project=PROJECT)
WEBHOOK_URL = os.environ.get("SLACK_WEBHOOK_URL", "")

def verificar_alertas(request=None):
    """Verifica umbrales críticos y envía notificaciones."""
```

```

alertas = []

# Alerta 1: Ventas muy por debajo del promedio
query = f"""
    SELECT AVG(ingresos) AS promedio, STDDEV(ingresos) AS desviacion
    FROM {DATASET}.metricas_diarias
    WHERE fecha >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
"""

stats = cliente.query(query).to_dataframe()
promedio = stats["promedio"].iloc[0]
desviacion = stats["desviacion"].iloc[0]

query_hoy = f"""
    SELECT ingresos FROM {DATASET}.metricas_diarias
    WHERE fecha = DATE_SUB(CURRENT_DATE(), INTERVAL 1 DAY)
"""

df_hoy = cliente.query(query_hoy).to_dataframe()

if len(df_hoy) > 0:
    ingresos_hoy = df_hoy["ingresos"].iloc[0]
    if ingresos_hoy < promedio - 2 * desviacion:
        alertas.append({
            "tipo": "ventas_bajas",
            "mensaje": f"Ventas de ayer ({ingresos_hoy:.2f}) "
                       f"muy por debajo del promedio ({promedio:.2f})"
        })

# Alerta 2: Sin pedidos en las últimas 24h
query_pedidos = f"""
    SELECT COUNT(*) AS pedidos FROM {DATASET}.orders
    WHERE order_date >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 24 HOUR)
"""

pedidos = cliente.query(query_pedidos).to_dataframe()
if pedidos["pedidos"].iloc[0] == 0:
    alertas.append({
        "tipo": "sin_pedidos",
        "mensaje": "No se registraron pedidos en las últimas 24 horas"
    })

# Enviar alertas
for alerta in alertas:
    print(f"ALERTA: {alerta['mensaje']}")
    # Aquí iría la lógica para enviar a Slack/Email

return json.dumps({"alertas": alertas, "total": len(alertas)})

```

Documentación del sistema

Cada componente del sistema debe estar documentado:

```
# Sistema de Reporting TechStore
```

```
## Componentes
1. BigQuery: techstore (dataset con tablas raw + derivadas)
2. Looker Studio: Dashboard Ejecutivo (enlace)
3. Pipeline Diario: Cloud Scheduler → Cloud Function → BigQuery
4. Alertas: Cloud Function (verifica métricas cada hora)

## Mantenimiento
- Pipeline diario se ejecuta a las 3:00 AM
- Dashboard se actualiza automáticamente
- Alertas se envían a Slack #techstore-analytics

## Costes estimados (mensuales)
- BigQuery storage: ~100 MB → ~$0.02
- BigQuery queries: ~50 GB procesados → ~$0.25
- Cloud Functions: 0 (free tier)
- Total estimado: < $1 USD/mes
```

Ejercicios

1. Crea la tabla `metricas\_diarias` en BigQuery con todos los KPIs diarios
2. Implementa el script `pipeline\_completo.py` con 3 transformaciones
3. ¿Cuál es la ventaja de tener tablas derivadas precalculadas?
4. Crea la tabla `rendimiento\_vendedores` con métricas de los últimos 90 días
5. Configura una alerta que se dispare si los ingresos de hoy son 0
6. Documenta cada componente de tu sistema de reporting
7. ¿Cómo estimarías el coste mensual de este sistema?
8. Añade una alerta que verifique si el número de pedidos supera un umbral máximo
9. Crea una tabla `top\_productos` actualizada semanalmente
10. Diseña un plan de mantenimiento para este sistema (frecuencia, responsables, backups)

Checkpoint 4: Pipeline Cloud Automatizado

Objetivo

Has llegado al final del libro. Este checkpoint integra todo lo aprendido: **implementar un pipeline cloud completamente automatizado** que actualice los datos de TechStore, ejecute transformaciones, genere informes y notifique al equipo — todo sin intervención manual.

Requisitos

- Proyecto GCP con BigQuery y Looker Studio configurados
- Checkpoints 1, 2 y 3 completados
- gcloud CLI instalado y autenticado
- Python con google-cloud-bigquery

Paso 1: Scheduled Query para resumen diario

Programa una consulta en BigQuery que se ejecute cada día a las 3:00 AM:

```
-- Configurar desde la consola de BigQuery
-- Nombre: resumen_diario_automatiko
-- Frecuencia: diario, 03:00
-- Destino: techstore.resumen_diario
-- Opción de escritura: WRITE_APPEND

SELECT
  CURRENT_DATE() AS fecha_generacion,
  DATE(order_date) AS fecha_venta,
  COUNT(*) AS pedidos,
  COUNT(DISTINCT customer_id) AS clientes_unicos,
  ROUND(SUM(total), 2) AS ingresos,
  ROUND(AVG(total), 2) AS ticket_promedio
FROM techstore.orders
WHERE order_date >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 DAY)
  AND order_date < TIMESTAMP_TRUNC(CURRENT_TIMESTAMP(), DAY)
GROUP BY fecha_venta;
```

Paso 2: Cloud Function para procesamiento adicional

Crea y despliega una Cloud Function que ejecute transformaciones adicionales:

```
"""
transformaciones_post/main.py
Ejecuta transformaciones después de la carga diaria.
"""

from google.cloud import bigquery

PROJECT = "techstore-analytics"
DATASET = "techstore"

cliente = bigquery.Client(project=PROJECT)

def ejecutar_transformaciones(request=None):
    """Ejecuta transformaciones post-carga."""
    queries = [
        f"""
        CREATE OR REPLACE TABLE {DATASET}.metricas_diarias AS
        SELECT
            DATE(order_date) AS fecha,
            COUNT(*) AS pedidos,
            COUNT(DISTINCT customer_id) AS clientes_activos,
            ROUND(SUM(total), 2) AS ingresos,
            ROUND(AVG(total), 2) AS ticket_promedio,
            ROUND(SUM(total) / NULLIF(COUNT(DISTINCT customer_id), 0), 2) AS ingresos_por_cliente
        FROM {DATASET}.orders
        GROUP BY fecha
        ORDER BY fecha
        """,
        f"""
        CREATE OR REPLACE TABLE {DATASET}.top_productos AS
        SELECT
            p.name,
            p.category,
            SUM(oi.quantity) AS unidades,
            ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
        FROM {DATASET}.order_items AS oi
        JOIN {DATASET}.products AS p ON oi.product_id = p.product_id
        WHERE oi.order_id IN (
            SELECT order_id FROM {DATASET}.orders
            WHERE order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
        )
        GROUP BY p.name, p.category
        ORDER BY ingresos DESC
        LIMIT 50
        """,
    ]

    for i, query in enumerate(queries, 1):
        job = cliente.query(query)
        job.result()
```

```
print(f"Transformación {i} completada: {job.total_bytes_processed / 1e6:.2f} MB")

return "Transformaciones completadas"
```

Despliega:

```
gcloud functions deploy transformaciones-post \
  --runtime python311 \
  --trigger-http \
  --allow-unauthenticated \
  --region us-central1
```

Paso 3: Cloud Scheduler para orquestar

Crea un job en Cloud Scheduler que ejecute la función después del scheduled query:

```
# Ejecutar transformaciones a las 4:00 AM (1 hora después del scheduled query)
gcloud scheduler jobs create http transformaciones-diarias \
  --schedule="0 4 * * *" \
  --uri="https://us-central1-PROJECT_ID.cloudfunctions.net/transformaciones-post" \
  --http-method=GET \
  --time-zone="America/Lima" \
  --location=us-central1
```

Paso 4: Función de informe semanal

Crea una función que genere un informe semanal en Cloud Storage:

```
"""
informe_semanal/main.py
Genera informe semanal en Excel y lo guarda en GCS.
"""

from google.cloud import bigquery, storage
from datetime import date, timedelta
import pandas as pd
import io

PROJECT = "techstore-analytics"
DATASET = "techstore"
BUCKET = "techstore-informes"

cliente_bq = bigquery.Client(project=PROJECT)
cliente_gcs = storage.Client()

def generar_informe_semanal(request=None):
    """Genera informe semanal de ventas."""
    hoy = date.today()
    semana_inicio = hoy - timedelta(days=7)

    # Consultas para el informe
    ventas = cliente_bq.query(f"""
```



```

SELECT
    DATE_TRUNC(order_date, WEEK(MONDAY)) AS semana,
    DATE(order_date) AS fecha,
    COUNT(*) AS pedidos,
    ROUND(SUM(total), 2) AS ingresos,
    ROUND(AVG(total), 2) AS ticket_promedio
FROM {DATASET}.orders
WHERE order_date >= '{semana_inicio.isoformat()}'
GROUP BY semana, fecha
ORDER BY fecha
""").to_dataframe()

top_productos = cliente_bq.query(f"""
    SELECT p.category, p.name, SUM(oi.quantity) AS vendidos,
           ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
    FROM {DATASET}.order_items AS oi
    JOIN {DATASET}.products AS p ON oi.product_id = p.product_id
    WHERE oi.order_id IN (
        SELECT order_id FROM {DATASET}.orders
        WHERE order_date >= '{semana_inicio.isoformat()}'
    )
    GROUP BY p.category, p.name
    ORDER BY ingresos DESC
    LIMIT 20
""").to_dataframe()

# Crear Excel en memoria
output = io.BytesIO()
with pd.ExcelWriter(output, engine="openpyxl") as writer:
    ventas.to_excel(writer, sheet_name="Ventas Diarias", index=False)
    top_productos.to_excel(writer, sheet_name="Top Productos", index=False)
output.seek(0)

# Subir a GCS
nombre_archivo = f"informe_semanal_{hoy.isoformat()}.xlsx"
bucket = cliente_gcs.bucket(BUCKET)
blob = bucket.blob(f"semanales/{nombre_archivo}")
blob.upload_from_string(output.getvalue(), content_type="application/vnd.openxmlformats-officedocument")

return f"Informe generado: gs://{BUCKET}/semanales/{nombre_archivo}"

```

Paso 5: Pipeline completo — Verificación

Crea un script que verifique el estado del pipeline completo:

```

"""
verificar_pipeline.py
Verifica que el pipeline completo funcione correctamente.
"""

from google.cloud import bigquery
from datetime import datetime, timedelta

```

```

PROJECT = "techstore-analytics"
DATASET = "techstore"
cliente = bigquery.Client(project=PROJECT)

print("=== Verificación del Pipeline TechStore ===")
print(f"Fecha: {datetime.now().isoformat()}")
print()

# 1. Verificar métricas diarias
try:
    df = cliente.query(f"""
        SELECT COUNT(*) AS dias, SUM(pedidos) AS total_pedidos,
            ROUND(SUM(ingresos), 2) AS total_ingresos
        FROM {DATASET}.metricas_diarias
        WHERE fecha >= DATE_SUB(CURRENT_DATE(), INTERVAL 7 DAY)
    """).to_dataframe()
    print(f" Métricas diarias: {df['dias'].iloc[0]} días, "
          f"{df['total_pedidos'].iloc[0]} pedidos, "
          f"${df['total_ingresos'].iloc[0]} ingresos (7 días)")
except Exception as e:
    print(f" Métricas diarias: ERROR – {e}")

# 2. Verificar top productos
try:
    df = cliente.query(f"""
        SELECT COUNT(*) AS productos FROM {DATASET}.top_productos
    """).to_dataframe()
    print(f" Top productos: {df['productos'].iloc[0]} productos registrados")
except Exception as e:
    print(f" Top productos: ERROR – {e}")

# 3. Verificar última ejecución del pipeline
try:
    df = cliente.query(f"""
        SELECT COUNT(*) AS pedidos_hoy
        FROM {DATASET}.orders
        WHERE DATE(order_date) = DATE_SUB(CURRENT_DATE(), INTERVAL 1 DAY)
    """).to_dataframe()
    print(f" Pedidos ayer: {df['pedidos_hoy'].iloc[0]}")
except Exception as e:
    print(f" Pedidos: ERROR – {e}")

print()
print("Verificación completada.")

```

Entregables del Checkpoint 4

Al completar este checkpoint deberías tener:

- [] Scheduled Query ejecutándose diariamente
- [] Cloud Function desplegada para transformaciones post-carga

- [] Cloud Scheduler orquestando la secuencia
- [] Función de informe semanal en Cloud Storage
- [] Dashboard en Looker Studio actualizado automáticamente
- [] Script de verificación del pipeline
- [] Documentación del sistema de reporting

Próximos pasos tras el checkpoint

1. **Expansión:** añade fuentes de datos adicionales (inventario, devoluciones)
2. **Alertas:** configura notificaciones a Slack/Email para fallos del pipeline
3. **Costes:** monitorea el gasto mensual con la consola de billing
4. **Colaboración:** invita a tu equipo al dashboard de Looker Studio
5. **Mejora continua:** ajusta las transformaciones según feedback del negocio

¡Felicidades! Has construido un sistema de reporting cloud completo, automatizado y profesional. Lo que antes requería hojas de cálculo manuales ahora se actualiza solo, todos los días, sin intervención humana. Bienvenido al mundo del analytics engineering en la nube.

Conclusión

De pandas a la nube

Has recorrido un camino impresionante. Empezaste con Python y pandas, limpiando datos y creando visualizaciones. Luego diste el salto a la nube con BigQuery, aprendiste a optimizar consultas con particiones y clustering, construiste dashboards profesionales en Looker Studio, y finalmente automatizaste todo con Cloud Functions y pipelines serverless.

Lo que has logrado:

1. **Python Avanzado:** merge, groupby, pivot, apply, visualización con matplotlib y seaborn, EDA completo
2. **Cloud Computing:** conceptos, GCP, autenticación, service accounts, gcloud CLI
3. **BigQuery:** data warehouse serverless, SQL analítico, nested fields, BQML, optimización con particiones y clustering
4. **Data Pipelines:** ETL/ELT, pipelines incrementales con checkpoint, orquestación
5. **Looker Studio:** dashboards interactivos, blending, campos calculados, parámetros
6. **Automatización:** Cloud Functions, Cloud Scheduler, Scheduled Queries, Workflows, Cloud Run
7. **Sistema Completo:** pipeline automatizado de reporting cloud con alertas y documentación

Habilidades adquiridas

Área	Skill
Lenguajes	Python (pandas, numpy, matplotlib, seaborn), SQL (BigQuery)
Cloud	GCP, BigQuery, Cloud Storage, Cloud Functions, IAM
BI	Looker Studio, dashboards, KPIs, storytelling con datos
Automatización	Cloud Scheduler, pipelines, alertas
Data Engineering	Arquitectura ELT, particionamiento, clustering, optimización de costes

¿Y ahora qué?

La colección **Data & Analytics Mastery** continúa. Este libro es el segundo de cinco. Dependiendo de tu camino, estos son los siguientes pasos:

Libro 3: Machine Learning y Ciencia de Datos con Python

Libro 4: Data Engineering y Arquitectura de Datos

Libro 5: Analytics Leadership y Storytelling

Mientras tanto, aquí tienes ideas para seguir practicando:

1. **Conecta una API real** (ej: datos climáticos, COVID, mercado financiero) a tu pipeline
2. **Expande TechStore** con más tablas (inventario, devoluciones, marketing)
3. **Implementa BQML** para predecir ventas futuras
4. **Crea un dashboard público** y compártelo en LinkedIn
5. **Automatiza todo** con Terraform para infraestructura como código

Reflexión final

La diferencia entre un analista de datos y un analista cloud-ready no está en las herramientas, sino en la mentalidad. Aprendiste a pensar en términos de escalabilidad, costes, automatización y colaboración. TechStore comenzó como un archivo SQLite en tu ordenador y terminó como un sistema cloud con dashboards en tiempo real, pipelines automatizados y alertas inteligentes.

Esa es la evolución que este libro quería lograr. No solo enseñarte herramientas, sino cambiar la forma en que piensas sobre los datos.

Gracias por acompañarme en este viaje. Nos vemos en el Libro 3: *Machine Learning y Ciencia de Datos con Python*.

— Alex Goyzueta Delgado

Apéndice A: Soluciones a los Ejercicios

Este apéndice contiene las soluciones a los ejercicios propuestos al final de cada capítulo. Las soluciones también están disponibles como archivos Python independientes en `codigos/soluciones/`.

Capítulo 1: Repaso de Python y pandas

Ejercicio 10 — EDA básico:

```
import pandas as pd
import sqlite3

conn = sqlite3.connect("../codigos/techstore.db")

# Ventas totales por categoría
df = pd.read_sql_query("""
    SELECT p.category, ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
    FROM order_items AS oi
    JOIN products AS p ON oi.product_id = p.product_id
    GROUP BY p.category
    ORDER BY ingresos DESC
""", conn)
print(df)

# Ticket promedio por cliente
df2 = pd.read_sql_query("""
    SELECT c.name, COUNT(o.order_id) AS pedidos, ROUND(AVG(o.total), 2) AS ticket_promedio
    FROM orders AS o
    JOIN customers AS c ON o.customer_id = c.customer_id
    GROUP BY c.name
    HAVING pedidos > 1
    ORDER BY ticket_promedio DESC
    LIMIT 10
""", conn)
print(df2)
conn.close()
```

Capítulo 2: pandas Avanzado

Ejercicio 8 — Merge orders + order_items + products:

```
import pandas as pd
```

```

import sqlite3

conn = sqlite3.connect("../codigos/techstore.db")

orders = pd.read_sql_query("SELECT * FROM orders", conn)
items = pd.read_sql_query("SELECT * FROM order_items", conn)
products = pd.read_sql_query("SELECT * FROM products", conn)

# Merge completo
df = orders.merge(items, on="order_id").merge(products, on="product_id")

# Top 10 productos por ingresos
top = (df.groupby(["product_id", "name"])
      .agg(ingresos=("unit_price", lambda x: (x * df.loc[x.index, "quantity"]).sum()),
          unidades=("quantity", "sum"))
      .sort_values("ingresos", ascending=False)
      .head(10))
print(top)
conn.close()

```

Capítulo 3: Limpieza de Datos

Ejercicio 7 — Pipeline de limpieza:

```

import pandas as pd

df = pd.read_csv("datos_ventas.csv")

# 1. Eliminar duplicados
df = df.drop_duplicates()

# 2. Manejar nulos
df["email"] = df["email"].fillna("sin_email@techstore.com")
df["phone"] = df["phone"].fillna("000-000-0000")

# 3. Estandarizar formatos
df["email"] = df["email"].str.lower().str.strip()
df["name"] = df["name"].str.title()

# 4. Eliminar outliers en precio (Z-score)
from scipy import stats
z_scores = stats.zscore(df["total"].dropna())
df = df[(abs(z_scores) < 3)]

print(f"Datos limpios: {len(df)} filas")

```

Capítulo 4: Visualización

Ejercicio 9 — Dashboard 2x2:

```

import matplotlib.pyplot as plt
import seaborn as sns

```

```

import pandas as pd
import sqlite3

conn = sqlite3.connect("../codigos/techstore.db")

orders = pd.read_sql_query("""
    SELECT DATE(order_date) AS fecha, total, status
    FROM orders
""", conn)
orders["fecha"] = pd.to_datetime(orders["fecha"])
orders["mes"] = orders["fecha"].dt.to_period("M").astype(str)

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Ventas por mes
ventas_mes = orders.groupby("mes")["total"].sum().sort_index()
ventas_mes.plot(ax=axes[0, 0], kind="line", marker="o", title="Ingresos Mensuales")

# 2. Distribución de totales
orders["total"].plot(ax=axes[0, 1], kind="hist", bins=30, title="Distribución de Totales")

# 3. Status de pedidos
orders["status"].value_counts().plot(ax=axes[1, 0], kind="bar", title="Estado de Pedidos")

# 4. Boxplot mensual
sns.boxplot(ax=axes[1, 1], data=orders, x="mes", y="total")
axes[1, 1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
conn.close()

```

Capítulo 5: EDA Completo

Ejercicio 10 — Insights de negocio:

```

import pandas as pd
import sqlite3

conn = sqlite3.connect("../codigos/techstore.db")

orders = pd.read_sql_query("SELECT * FROM orders", conn)
customers = pd.read_sql_query("SELECT * FROM customers", conn)
items = pd.read_sql_query("SELECT * FROM order_items", conn)
products = pd.read_sql_query("SELECT * FROM products", conn)

# Ticket promedio por ciudad
df = orders.merge(customers, on="customer_id")
top_ciudades = df.groupby("city").agg(
    pedidos=("order_id", "count"),
    ingresos=("total", "sum"),
    ticket_promedio=("total", "mean")
)

```



```

).sort_values("ingresos", ascending=False).head(10)
print("Top 10 ciudades por ingresos:")
print(top_ciudades)

# Productos más rentables
df2 = items.merge(products, on="product_id")
top_productos = df2.groupby("name").agg(
    unidades=("quantity", "sum"),
    ingresos=("unit_price", lambda x: (x * df2.loc[x.index, "quantity"]).sum())
).sort_values("ingresos", ascending=False).head(10)
print("\nTop 10 productos por ingresos:")
print(top_productos)

# Clientes con mayor recurrencia
df3 = df.groupby("customer_id").agg(
    nombre=("name", "first"),
    pedidos=("order_id", "count"),
    total=("total", "sum"),
    primera_compra=("order_date", "min"),
    ultima_compra=("order_date", "max")
)
df3["dias_entre"] = (pd.to_datetime(df3["ultima_compra"]) - pd.to_datetime(df3["primera_compra"])).dt.days
print("\nClientes más recurrentes:")
print(df3.sort_values("pedidos", ascending=False).head(10))
conn.close()

```

Capítulo 6: Cloud Computing

Ejercicio 2 — Costes BigQuery:

- Almacenamiento BigQuery (región US): \$0.02 por GB/mes (datos activos), \$0.01 por GB/mes (datos de larga duración)
- Consultas (on-demand): \$5 por TB procesado
- Free tier: 10 GB de almacenamiento y 1 TB de consultas por mes

Capítulo 7: GCP Configuración

Ejercicio 10 — gcloud query:

```
bq query "SELECT CURRENT_DATE() AS hoy"
```

Capítulo 8: BigQuery

Ejercicio 6 — INSERT en tabla test:

```

CREATE TABLE techstore.test (
    id INT64 NOT NULL,
    nombre STRING(100),
    fecha DATE
);

```

```

INSERT INTO techstore.test (id, nombre, fecha) VALUES
  (1, 'Cliente A', '2024-01-15'),
  (2, 'Cliente B', '2024-02-20'),
  (3, 'Cliente C', '2024-03-10');

SELECT * FROM techstore.test;

DROP TABLE techstore.test;

```

Capítulo 9: BigQuery Avanzado

Ejercicio 5 — Comparar escaneo (con/sin partición):

```

-- Sin partición: escanea toda la tabla
SELECT COUNT(*) FROM techstore.orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31';

-- Con partición: escanea solo 3 meses
SELECT COUNT(*) FROM techstore.orders_partitioned
WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31';

```

Capítulo 10: Python + BigQuery

Ejercicio 6 — Función con parámetro fecha:

```

from google.cloud import bigquery

def ventas_del_mes(anio, mes):
    cliente = bigquery.Client()
    query = f"""
        SELECT DATE_TRUNC(order_date, MONTH) AS mes,
               COUNT(*) AS pedidos,
               ROUND(SUM(total), 2) AS ingresos
        FROM techstore.orders
        WHERE EXTRACT(YEAR FROM order_date) = {anio}
              AND EXTRACT(MONTH FROM order_date) = {mes}
        GROUP BY mes
    """
    df = cliente.query(query).to_dataframe()
    return df

print(ventas_del_mes(2024, 6))

```

Capítulo 11: SQL Analítico y Nested Fields

Ejercicio 3 — Percentiles:

```

SELECT
  APPROX_QUANTILES(total, 100)[OFFSET(10)] AS p10,
  APPROX_QUANTILES(total, 100)[OFFSET(50)] AS p50_mediana,
  APPROX_QUANTILES(total, 100)[OFFSET(90)] AS p90,

```

```
MAX(total) AS maximo
FROM techstore.orders;
```

Capítulo 12: Data Pipelines

Ejercicio 3 — Checkpoint:

Un checkpoint registra la última posición procesada (ej: última fecha, último ID). En pipelines incrementales evita reprocesar datos ya cargados, ahorrando tiempo y costes.

Capítulo 13: Looker Studio

Ejercicio 7 — Consulta personalizada en Looker Studio:

```
SELECT
  DATE_TRUNC(order_date, MONTH) AS mes,
  COUNT(DISTINCT customer_id) AS clientes_activos,
  ROUND(SUM(total), 2) AS ingresos,
  COUNT(*) AS pedidos
FROM techstore.orders
WHERE order_date >= '2024-01-01'
GROUP BY mes
ORDER BY mes;
```

Capítulo 14: Looker Studio Avanzado

Ejercicio 1 — Campo calculado de categorización:

```
CASE
  WHEN total < 200 THEN 'Pequeño'
  WHEN total < 500 THEN 'Mediano'
  WHEN total < 1000 THEN 'Grande'
  ELSE 'Premium'
END
```

Capítulo 15: Cloud Functions

Ejercicio 3 — Función que consulta BigQuery:

```
from google.cloud import bigquery
from datetime import date

def pedidos_hoy(request):
    cliente = bigquery.Client()
    hoy = date.today().isoformat()
    query = f"""
        SELECT COUNT(*) AS pedidos, ROUND(SUM(total), 2) AS ingresos
        FROM techstore.orders
        WHERE DATE(order_date) = '{hoy}'
    """
    df = cliente.query(query).to_dataframe()
    return {"fecha": hoy, "pedidos": int(df["pedidos"].iloc[0]),
```

```
"ingresos": float(df["ingresos"].iloc[0])}
```

Capítulo 16: Schedule Queries

Ejercicio 2 — Diferencia Cloud Scheduler vs Scheduled Queries:

- Scheduled Queries: solo ejecutan SQL en BigQuery, no requieren código
- Cloud Scheduler: dispara cualquier servicio (HTTP, Pub/Sub, App Engine), más flexible

Capítulo 17: Proyecto Final

Ejercicio 5 — Alerta de ingresos cero:

```
def verificar_ingresos_cero():
    cliente = bigquery.Client()
    df = cliente.query("""
        SELECT COUNT(*) AS pedidos FROM techstore.orders
        WHERE DATE(order_date) = DATE_SUB(CURRENT_DATE(), INTERVAL 1 DAY)
    """).to_dataframe()
    if df["pedidos"].iloc[0] == 0:
        print("ALERTA: No se registraron pedidos ayer")
        # Enviar notificación...
```

Apéndice B: Cheatsheets

pandas

```
import pandas as pd

# Lectura
df = pd.read_csv("archivo.csv")
df = pd.read_excel("archivo.xlsx")
df = pd.read_sql_query("SELECT * FROM tabla", conexion)

# Exploración
df.head(10)          # Primeras filas
df.info()            # Tipos y nulos
df.describe()        # Estadísticas
df.shape             # Dimensiones
df.columns           # Nombres columnas
df.dtypes            # Tipos
df.isnull().sum()    # Nulos por columna

# Selección
df["columna"]        # Una columna
df[["a", "b"]]       # Varias columnas
df.iloc[0:5]         # Por posición
df.loc[df["x"] > 5]   # Por condición

# Limpieza
df.drop_duplicates()
df.dropna()
df.fillna(valor)
df.rename(columns={"old": "new"})
df["col"] = df["col"].astype(float)

# Transformación
df.groupby("col").agg({"x": "sum", "y": "mean"})
df.pivot_table(index="a", columns="b", values="c", aggfunc="sum")
df.merge(df2, on="key", how="left")
df.sort_values("col", ascending=False)
df["nueva"] = df["a"].apply(lambda x: x * 2)

# Fechas
pd.to_datetime(df["fecha"])
```

```
df["mes"] = df["fecha"].dt.month
df["año"] = df["fecha"].dt.year
```

BigQuery SQL

```
-- Particionamiento
CREATE TABLE dataset.tabla
PARTITION BY DATE_TRUNC(col_fecha, MONTH)
AS SELECT * FROM origen;

-- Clustering
CREATE TABLE dataset.tabla
PARTITION BY DATE_TRUNC(col_fecha, MONTH)
CLUSTER BY col1, col2
AS SELECT * FROM origen;

-- Nested fields
SELECT customer.name, customer.city
FROM dataset.orders_nested;

-- UNNEST arrays
SELECT * FROM dataset.tabla,
UNNEST(col_array) AS elemento;

-- BQML
CREATE MODEL dataset.modelo
OPTIONS(model_type='linear_reg',
        input_label_cols=['target'])
AS SELECT * FROM dataset.tabla;

-- Scheduled Query (desde consola)
-- Configurar: frecuencia, destino, WRITE_APPEND/TRUNCATE

-- Scripting
DECLARE var DATE DEFAULT '2024-01-01';
EXECUTE IMMEDIATE "SELECT @var";

-- INFORMATION_SCHEMA
SELECT * FROM region-us.INFORMATION_SCHEMA.JOBS;
SELECT * FROM dataset.INFORMATION_SCHEMA.TABLES;
```

gcloud CLI

```
# Autenticación
gcloud auth login
gcloud auth application-default login
gcloud config set project PROYECTO_ID

# BigQuery
bq ls                                # Listar datasets
```

```

bq show dataset.tabla      # Info de tabla
bq query "SELECT 1"        # Ejecutar SQL
bq query --dry_run "SQL"   # Estimar coste
bq mk dataset              # Crear dataset
bq rm -r dataset           # Eliminar dataset

# Cloud Functions
gcloud functions deploy NOMBRE \
  --runtime python311 \
  --trigger-http \
  --allow-unauthenticated \
  --region us-central1

gcloud functions logs read NOMBRE
gcloud functions delete NOMBRE

# Cloud Scheduler
gcloud scheduler jobs create http NOMBRE \
  --schedule="0 6 * * *" \
  --uri="URL" \
  --http-method=GET

# Cloud Storage
gcloud storage cp archivo.csv gs://bucket/
gcloud storage ls gs://bucket/

```

Looker Studio

```

// Campos calculados
SUM(total) / COUNT(order_id)
CASE WHEN x > 10 THEN 'Alto' ELSE 'Bajo' END
FORMAT_DATETIME('%Y-%m', order_date)
DATE_DIFF(CURRENT_DATE(), fecha, YEAR)

// Parámetros
// Crear parámetro: nombre, tipo, valor default
// Usar: SUM(total) / parametro_objetivo * 100

// Blending
// Añadir datos → Combinar datos
// Seleccionar clave de unión
// Elegir dimensiones y métricas de cada fuente

```

Cloud Functions

```

# Función HTTP
def mi_funcion(request):
    nombre = request.args.get("nombre", "Mundo")
    return f"Hola, {nombre}"

```

```

# Función Storage
def procesar_archivo(event, context):
    archivo = event["name"]
    bucket = event["bucket"]
    print(f"Archivo: gs://{bucket}/{archivo}")

# Función Pub/Sub
def procesar_mensaje(event, context):
    import base64, json
    datos = base64.b64decode(event["data"]).decode("utf-8")
    mensaje = json.loads(datos)
    print(f"Mensaje: {mensaje}")

```

Conceptos cloud

Concepto	Definición
IaaS	Infraestructura como servicio (VMs, redes)
PaaS	Plataforma como servicio (BigQuery, App Engine)
SaaS	Software como servicio (Looker Studio, Gmail)
Serverless	Código sin gestión de servidores
Data Warehouse	Datos estructurados para análisis
Data Lake	Datos brutos en formato nativo
ETL	Extract → Transform → Load
ELT	Extract → Load → Transform
IAM	Gestión de identidades y accesos
SLA	Acuerdo de nivel de servicio
Cold start	Latencia inicial en funciones serverless

Apéndice C: Configuración de GCP y Costes

Free Tier de Google Cloud

Google Cloud ofrece un **free tier** generoso que cubre la mayoría de lo que necesitas para este libro:

Servicio	Free Tier	Límite mensual
BigQuery	10 GB de almacenamiento	Gratis
BigQuery	1 TB de consultas procesadas	Gratis
Cloud Functions	2 millones de invocaciones	Gratis
Cloud Functions	400,000 GB-segundos de computación	Gratis
Cloud Storage	5 GB por región	Gratis
Cloud Scheduler	3 trabajos programados	Gratis
Looker Studio	Ilimitado	Gratis

Además, al crear tu cuenta obtienes **\$300 USD en créditos** para usar durante 90 días.

Costes estimados del proyecto TechStore

Con los datos de TechStore (~6000 pedidos, ~500 productos, ~250 clientes):

Concepto	Estimación mensual
Almacenamiento BigQuery	~100 MB → < \$0.01
Consultas (aprendizaje)	~2 GB/mes → < \$0.01
Cloud Functions (pruebas)	~1000 invocaciones → \$0.00
Cloud Storage (informes)	~10 MB → \$0.00
Total estimado	**< \$0.50 USD/mes**

Cómo monitorear costes

Alertas de presupuesto

1. Ve a `console.cloud.google.com/billing``
2. "Presupuestos y alertas" "Crear presupuesto"
3. Nombre: "TechStore Analytics"
4. Monto: \$5 USD/mes
5. Alertas: 50%, 90%, 100%

6. Notificaciones: Email a tu cuenta

Recomendaciones para ahorrar costes

1. **Siempre previsualiza:** usa ``LIMIT`` o ``` para estimar coste antes de ejecutar
2. **Elimina tablas temporales:** borra datasets de prueba cuando termines
3. **Particiona y agrupa:** reduce datos escaneados hasta 90%
4. **Evita SELECT *:** selecciona solo columnas necesarias
5. **Usa la consola de facturación:** revisa semanalmente
6. **Pausa recursos no usados:** detén VMs, elimina funciones de prueba
7. **Usa committed use discounts:** si tienes carga predecible (más avanzado)

Troubleshooting común

Error: "Billing account not found"

- Solución: verifica que la facturación esté habilitada en ``console.cloud.google.com/billing``

Error: "Access Denied: BigQuery BigQuery"

- Solución: habilita la API de BigQuery en "APIs y servicios" "Biblioteca"

Error: "Dataset not found in location"

- Solución: verifica que el dataset existe y estás en la región correcta

Error: "Exceeded rate limits"

- Solución: espera unos segundos y reintenta. Reduce la concurrencia

Error: "401 Unauthorized" en gcloud

- Solución: ejecuta ``gcloud auth login`` y ``gcloud auth application-default login``

Error: Cloud Function deployment timeout

- Solución: reduce dependencias, verifica que requirements.txt esté completo

Regiones recomendadas

Región	Descripción	Coste
<code>`us-central1`</code>	Iowa, USA	Más barato
<code>`us-east1`</code>	Carolina del Sur, USA	Barato
<code>`europe-west1`</code>	Bélgica	Bueno para Europa
<code>`southamerica-east1`</code>	São Paulo, Brasil	Más caro
<code>`asia-southeast1`</code>	Singapur	Bueno para Asia

Para este libro, usa ``us-central1`` (menor coste, buena latencia global).

Recursos adicionales

- **Documentación BigQuery:** <https://cloud.google.com/bigquery/docs>
- **Pricing Calculator:** <https://cloud.google.com/products/calculator>

- **Google Cloud Skills Boost:** <https://www.cloudskillsboost.google/>
- **BigQuery public datasets:** <https://cloud.google.com/bigquery/public-data>
- **Looker Studio Help:** <https://support.google.com/looker-studio>

Sobre el Autor

Alex Goyzueta Delgado es analista de datos, instructor y emprendedor tecnológico con más de una década de experiencia transformando datos en decisiones de negocio.

Su trayectoria abarca desde bases de datos relacionales y hojas de cálculo hasta plataformas cloud empresariales, trabajando con startups y corporaciones en Latinoamérica para implementar soluciones de analytics que generan impacto real.

Fundador de **Data & Analytics Academy**, donde ha formado a más de 5000 estudiantes en análisis de datos, SQL, Python y cloud computing a través de cursos prácticos y project-based learning.

Cree firmemente en que la mejor forma de aprender datos es construyendo proyectos reales desde el día uno. Su metodología combina fundamentos técnicos sólidos con casos de negocio prácticos, preparando a sus estudiantes no solo para usar herramientas, sino para pensar como analistas.

Este libro es el segundo de la colección **Data & Analytics Mastery**, una serie diseñada para llevar al lector desde cero hasta analytics professional en la nube.

Conéctate con el autor:

- LinkedIn: https://www.linkedin.com/in/alexgoyzueta/python_data_cloud
- GitHub: <https://github.com/alexgoyzueta>

Otros libros de la colección:

1. *Los Fundamentos del Analista de Datos* — Domina SQL, Excel y Python desde cero
2. *Python para Datos y tu Primer Data Warehouse Cloud* — Este libro
3. *Machine Learning y Ciencia de Datos con Python* — Próximamente
4. *Data Engineering y Arquitectura de Datos* — Próximamente
5. *Analytics Leadership y Storytelling* — Próximamente